



Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Estado de México

CRISP-DM Evaluación e Implementación de Soluciones con LLMs

TC3007C. Inteligencia artificial avanzada para la ciencia de datos II

Grupo: 501

Equipo 2:

Alan Contreras Prieto - A01749667

Alan Rodrigo Vega Reza - A01750658

Daniel Kaled Bernal Ayala - A01750047

Ameyalli Contreras Sánchez - A01749075

José Antonio Moreno Tahuilán - A01747922

Iván Alexander Ramos Ramírez - A01750817

Yóse Miguel Sotomayor Carneado - A01750908

Carlos Alberto Zamudio Velázquez - A01799283

Prof. Alberto Michel Pérez Domínguez

Fecha de entrega: 05 de diciembre de 2025

Semestre agosto - diciembre 2025

Índice

1. Determinación de objetivos de negocio.....	6
1.1. Antecedentes / Contexto.....	6
1.2. Objetivos del negocio.....	7
1.2.A. Elevator Pitch del Producto (Visión Sintetizada).....	8
1.3. Criterios de éxito.....	7
2.1. Inventario de recursos.....	9
2.2. Requisitos, supuestos y restricciones.....	9
2.2.1. Requisitos.....	9
2.2.1.1. Funcionales.....	9
2.2.1.2. No Funcionales.....	10
2.2.2. Supuestos.....	10
2.2.3. Restricciones.....	10
2.3. Riesgos y contingencias.....	11
2.4. Terminología.....	12
2.5. Costos y Beneficios.....	12
2.5.1. Costos.....	12
2.5.1.1. Costo total:.....	13
2.5.2. Beneficios.....	13
3. Determinación de objetivos de minería de datos.....	14
3.1 Objetivos de Minería de Datos.....	14
3.2. Criterios de Éxito.....	14
4. Producción del plan del proyecto.....	15
4.1. Plan del proyecto.....	15
4.2. Evaluación inicial de herramientas y técnicas.....	16
4.2.1. Lista de herramientas.....	16
4.2.2. Lista de técnicas (provisional).....	17
4.2.3. Diagramas de arquitectura final.....	18
4.2.4. Diagrama de infraestructura final.....	20
5. Entendimiento de los datos.....	20
6. Selección de Datos.....	20
6.1. Tabla Articles.....	20
6.2. Tabla Customers.....	22
6.3. Tabla Transactions.....	22
6.4. Final Shapes.....	23
7. Limpieza de Datos.....	23
7.1. Valores faltantes.....	23
7.1.1. Tabla Articles.....	23
7.1.2. Tabla Customers.....	24
7.2. Valores atípicos.....	26

7.3. Ruido e inconsistencias.....	27
8. Construcción de Datos.....	28
8.1. Creación de Features.....	28
8.1.1. Precio por artículo.....	28
8.1.2. Variables temporales.....	29
8.1.3. Variables de comportamiento.....	31
8.1.4. Variables de producto.....	32
8.2. Transformaciones.....	33
8.2.1. Log-transform + Estandarización.....	33
8.2.2. Estandarización.....	33
8.2.3. Min-Max.....	33
8.3. Derivación de Features.....	34
8.3.1. Variables Binarias.....	34
8.3.2. Discretización.....	34
8.3.3. Codificación de variables categóricas.....	34
8.3.3.1. <i>One-Hot encoding</i>	34
8.3.3.2. <i>Label encoding</i>	34
9. Integración de Datos.....	35
9.1. Claves utilizadas.....	35
9.2. Tipo de unión.....	35
9.3. Verificación de integridad de los datos.....	36
10. Formateo de Datos.....	37
10.1. Estructura.....	37
10.2. Tipos de datos.....	38
10.3. Conjuntos de entrenamiento, validación y prueba.....	38
10.3.1. Sets Final Shapes.....	38
10.4. Código empleado.....	39
11. Selección de Técnica de Modelado.....	39
11.1. Análisis de Algoritmos Candidatos.....	39
11.1.1. ALS (Alternating Least Squares).....	39
11.1.2. Whisper: Reconocimiento de Voz.....	39
11.1.3. LangGraph + OCI GenAI: Agente Conversacional.....	40
11.1.4. LLMs para Generación de Reviews Sintéticas (Meta llama).....	40
11.1.5. Búsqueda semántica de reviews (SentenceTransformers + FAISS)	40
11.1.6. ChatGPT (OpenAI).....	41
11.2. Selección Final y Justificación.....	41
11.2.1. ALS.....	41
11.2.1.1. <i>¿Por qué fue seleccionada?</i>	41
11.2.1.2. <i>Trade-offs considerados</i>	41
11.2.2. Whisper – fast, Speech-to-Text.....	41
11.2.2.1. <i>¿Por qué fue seleccionada?</i>	41

11.2.2.2. Trade-offs considerados.....	42
11.2.3. Agente Multimodal con LangGraph + OCI GenAI.....	42
11.2.3.1. ¿Por qué fue seleccionada?.....	42
11.2.3.2. Trade-offs considerados.....	42
11.2.4. LLM para Generación de Reviews Sintéticas.....	42
11.2.4.1. ¿Por qué fue seleccionada?.....	42
11.2.4.2. Trade-offs considerados.....	43
11.2.5. Búsqueda semántica con Embeddings + FAISS.....	43
11.2.5.1. ¿Por qué fue seleccionada?.....	43
11.2.5.2. Trade-offs considerados.....	43
11.3. Supuestos del Modelo.....	43
11.3.1. ALS.....	43
11.3.2. Whisper: Reconocimiento de Voz.....	44
11.3.3. Agente Multimodal (LangGraph + OCI GenAI).....	44
11.3.4. LLM para generación de reviews.....	44
11.3.5. Clustering de Reviews (UMAP + HDBSCAN).....	45
12. Diseño de pruebas.....	45
12.2. Métricas de Evaluación.....	45
12.2.1. MAP@K (Mean Average Precision @K).....	45
12.2.2. Word Error Rate (WER).....	46
12.2.3. Intent Accuracy.....	46
12.2.4. Semantic Similarity Score (cosine similarity).....	46
12.2.5. Calidad de Generación (LLMs – Reviews Sintéticas).....	46
12.3. Criterios de Éxito.....	47
13. Construcción del Modelo.....	47
13.1. Modelos NO-LLM.....	47
13.1.1. ALS (Sistema de Recomendaciones).....	47
13.1.1.1. Configuración de Parámetros.....	47
13.1.1.2. Proceso de Entrenamiento.....	47
13.1.1.3. Modelo Resultante.....	48
13.1.2. Búsqueda semántica de reviews con FAISS.....	48
13.1.2.1. Configuración de Parámetros.....	48
13.1.2.2. Proceso de entrenamiento.....	48
13.1.2.3. Modelo Resultante.....	48
13.1.3. Whisper (Speech-to-Text).....	49
13.1.3.1. Parámetros.....	49
13.1.3.2. Proceso de entrenamiento.....	49
13.1.3.3. Modelo Resultante.....	49
13.2. Modelos LLM.....	49
13.2.1. Grok-4 Fast (Conversación + Clasificación).....	49
13.2.1.1. Selección del Modelo Base.....	49

13.2.1.2 Estrategia de Prompting.....	49
13.2.1.3 Evaluación.....	49
13.2.2. Llama 3.2 90B Vision (Análisis de Imágenes).....	49
13.2.2.1. Selección.....	49
13.2.2.2. Prompting.....	50
13.2.3. Llama-4 Maverick 17B (Generación Masiva de Reviews Sintéticas).....	50
13.2.3.1. Selección.....	50
13.2.3.2. Prompting.....	50
13.2.3.3 Evaluación.....	50
14. Evaluación del Modelo.....	50
14.1. Resultados de Evaluación:.....	50
14.1.1. Evaluación a ALS.....	50
14.1.1.1. Resultados de Evaluación a ALS.....	50
14.2.1. Reviews sintéticas.....	51
14.1.3. Evaluación a Whisper.....	51
14.1.3.1. Resultados de la evaluación a Whisper.....	51
14.1.4. Evaluación a Grok.....	51
14.1.4.1. Resultados de la evaluación a Grok.....	51
14.1.5. Evaluación a FAISS.....	52
14.1.5.1. Resultado de la evaluación a FAISS.....	52
14.2. Análisis Crítico.....	52
14.3. Conclusiones y Recomendaciones.....	53
15. Evaluación y Despliegue.....	53
15.1. Viabilidad práctica.....	53
15.2. Rentabilidad.....	54
15.2.1. Material promocional.....	54
15.3. Impacto empresarial.....	55
Anexo A. Definición de Pruebas de usabilidad.....	55
Anexo B. Reporte de uso de IA.....	55
Referencias.....	56

1. Determinación de objetivos de negocio.

1.1. Antecedentes / Contexto

Nuestra tienda busca integrarse al crecimiento del e-commerce en México, que de acuerdo con AMVO (Asociación Mexicana de Venta Online), el sector creció un 24.6% en 2023. Sin embargo, en un mercado dominado por gigantes como Bodega Aurrerá, Amazon, Mercado Libre, Walmart, Coppel, Oxxo o Liverpool, la diferenciación e innovación es clave para competir. Pero sabe que en un mercado tan competitivo, es necesario tener un diferenciador. El principal reto actual es que los consumidores demandan experiencias rápidas, confiables y fáciles de usar. De no ser así, abandonan la compra y la experiencia no se vuelve cómoda para ellos.

El principal reto es que los consumidores demandan experiencias rápidas y confiables de usar como menciona AMVO en su estudio del 2024. Este menciona lo siguiente: “Las razones para comprar en el canal digital recaen en la comodidad y facilidad de compra (81%), en los beneficios logísticos (67%), en la variedad y disponibilidad (67%), así como en los beneficios adicionales y de pago (64%). (AMVO, 2024)”. Dando así los principales enfoques que se consideran como puntos importantes en el desarrollo del proyecto.

Los principales Stakeholders que encontramos son:

Stakeholder	Interés principal	Decisión operativa que soporta
Clientes finales	Comprar de manera rápida, confiable, segura y fácil, con precios competitivos.	Definir prioridades en la experiencia del usuario como lo es facilidad de pago con diferentes métodos, tiempos de entrega, políticas de devolución.
Empresa	Aumentar la tasa de conversión, menos devoluciones y mayor participación en el mercado.	Determinan la estrategia comercial y de inversión tecnológica.
Proveedores de producto	Tener visibilidad y rotación de sus productos en la tienda online.	Afectan las decisiones de catálogo, surtido, precios y promociones.
Proveedores logísticos	Mantener relaciones estables con el retailer y cumplir con entregas.	Impactan directamente en tiempos y costos de entrega, lo que afecta la satisfacción del cliente y la retención.
Equipo de desarrollo	Actualización de la aplicación para evitar errores.	Impactan en el correcto funcionamiento de la aplicación y su adecuado mantenimiento para no arruinar la experiencia del usuario.

Actualmente, el desafío es transformarse en una plataforma digital atractiva, usable y diferenciada, que no solo capture más ventas, sino que aumente la satisfacción del cliente y genere lealtad en un entorno altamente competitivo.

Id	1.2. Objetivos del negocio ¿Qué queremos lograr para el negocio/dominio?	1.3. Criterios de éxito Métricas cuantificables y específicas que permitirán evaluar si se han cumplido los objetivos de negocio.
NP.001	Incrementar las ventas totales del negocio en un 15% en un periodo de 6 meses.	Observar el porcentaje de venta para asegurar el crecimiento constante mensualmente.
NP.001. 1	Por medio del agente de IA se plantea acompañar al usuario en el proceso de compra mediante interacciones cercanas y búsqueda de productos mediante reconocimiento de imágenes relacionadas a sus preferencias para incrementar las ventas mencionadas en un 10%.	Aumentar las ventas asistidas por agentes/chatbots un 10% respecto al promedio previo en un periodo de 3 meses.
NP.001. 2	Se busca incrementar el otro 5% de las ventas totales mediante un sistema de notificaciones y recomendaciones relacionadas a los descuentos que existen en los productos deseados por los usuarios.	● Conseguir un índice de atención de notificaciones y recomendaciones del 15% ● Conseguir que el 5% de las notificaciones y recomendaciones atendidas terminen en compra.
NP.002	Maximizar la cantidad de ventas de productos que entran en el carrito en un 20% en el primer trimestre de despliegue.	Aumentar la venta completa del carrito en un 20% quincenalmente.
NP.003	Mejorar la satisfacción del cliente al usar la página web en un 15% durante el primer trimestre del despliegue.	Aumentar los indicadores de satisfacción del cliente como NPS (Net Promoter Score) o CSAT (Customer Satisfaction Score) un 15% en un periodo de 6 meses.
NP.003. 1	Facilitar la búsqueda de productos mediante las interacciones con el chat que minimicen la cantidad de clics en un 20%, para que así puedan encontrar posibles productos relacionados a sus deseos/intenciones para mejorar la satisfacción.	Disminuir clics necesarios para: ● Agregar producto al carrito con chat → 0 clics ● Realizar compra asistida por chat → 1 clic para confirmar

NP.004	Disminuir la tasa de abandono 10% de usuarios que lleven más de 1 mes sin visitar el sitio web.	Reducir la cantidad de usuarios que mantengan inactiva su cuenta durante más de 1 mes en 5%..
--------	---	---

1.2.A. Elevator Pitch del Producto (Visión Sintetizada)

- **Para:** los usuarios que busquen una experiencia nueva en el mercado y quieran agilizar sus compras en línea.
- **Que tienen:** dificultad para realizar compras en línea o se vuelve muy engorroso el proceso. Usuarios que se frustran con búsquedas largas, confusas y sin encontrar resultados compatibles con sus intereses.
- **Nombre producto:** Zenith
- **Nuestro producto/proyecto es una:** plataforma con un agente inteligente de IA.
- **Que proporciona:** búsquedas recomendadas y personalizadas para aumentar la agilidad al elegir y comprar productos. Se caracteriza por la búsqueda del producto del cliente mediante texto e imágenes, considerando igual las reseñas de los productos para ofrecerle al usuario algo adaptado a su gusto y necesidad. Además, contará con un sistema de notificaciones personalizadas sobre descuentos de productos que desee o se asemeje a lo que quiere el usuario.
- **A diferencia de:** Amazon, Mercado Libre, Liverpool.
- **Nuestra solución:** La solución que ofrecemos busca transformar la experiencia de compra en línea a través de un acompañamiento a modo de un amigo que pueda saber de tus gustos y de tus necesidades. El agente inteligente ajusta en tiempo real las recomendaciones en función de las preferencias, comportamientos y análisis de imágenes para mejorar la precisión de los resultados y la relevancia de estos. Al contrario de las plataformas que únicamente listan productos, nuestro asistente guía a los usuarios en el proceso de compra, lo cual significa que el asistente disminuye la fricción y ayuda a los usuarios en la toma de decisiones rápidas y satisfactorias.

Valor general: El cliente logra de manera fácil y segura la compra de sus productos. Aunado a esto, buscamos que la empresa logre mayor tasa de conversión, haciendo que disminuyan las devoluciones. Además, los proveedores deben de estar satisfechos en tanto a la venta de su producto como la agilidad de los mismos para realizar entregas. Por último, el equipo de desarrollo debe de estar actualizado y capacitado para mejorar la aplicación y dar mantenimiento de manera correcta si así lo requiere.

2. Evaluación de la situación

2.1. Inventario de recursos

- Base de datos proporcionada por el socio formador (Oracle).
- Créditos proporcionados para utilizar servicios de AWS.
- Créditos proporcionados para utilizar servicios de Oracle.
- Software para manejo de lenguajes de programación:
 - VSCode
 - Software para manejo de los datos:
 - Oracle tools
- Hardware: Aproximadamente 8 equipos de computo
 1. MacBook Pro, 24 GB de RAM, Chip M4, 1.4 TB
 2. Linux Pop OS, 96 GB de RAM, Intel i9, 2 TB, NVIDIA 4090 RTX
 3. Windows, 32 GB de RAM, Intel i7, 1 TB, NVIDIA 2060 RTX
 4. Windows, 17 GB de RAM, Intel i7, 1 TB, NVIDIA 3070 RTX
 5. Windows, 32 GB de RAM Ryzen 5 5600x, 2.5 TB NVIDIA 4060 RTX
 6. Windows, 32 GB de RAM, Intel i7, 1 TB, NVIDIA 4070 RTX
 7. Windows, 32 GB de RAM, Intel i7, 1 TB, NVIDIA 4070 RTX
 8. Windows 128 GB de RAM, Ryzen 9, 4TB, NVIDIA 5090 RTX
- Personal
 - 1 Ing. Robótico y Sistemas Digitales
 - 3 Ing. Ciencia de datos
 - 4 Ing. Tecnologías computacionales

2.2. Requisitos, supuestos y restricciones

2.2.1. Requisitos

2.2.1.1. Funcionales

Del sistema:

1. El sistema debe dar recomendaciones de producto al usuario tomando como base el producto que está observando en ese momento (prioritario).
2. El sistema debe permitir al usuario buscar en la tienda y visualizar los productos.
3. El sistema debe permitir al usuario interactuar con un chatbot usando un LLM para buscar productos en base a las características deseadas por el usuario.
4. El sistema debe permitir que el usuario cree una cuenta con un nombre de usuario y un método de pago.
5. El sistema debe dar recomendaciones de producto al usuario tomando como base su historial de compra.
6. El sistema debe dar recomendaciones de producto al usuario tomando como base el carrito de compras del usuario.
7. El sistema debe dar recomendaciones de producto al usuario usando de referencia las reviews de los productos.

8. El sistema debe dar recomendaciones de producto al usuario tomando como base el rating de los productos dentro de la tienda.
9. El sistema debe permitir al usuario agregar productos a un carrito de compra.
10. El sistema debe permitir al usuario pagar por los productos que estén en el carrito de compra.
11. El sistema debe permitir al usuario comprar productos de manera automática a través del chatbot.
12. El sistema debe ser capaz de mostrar los últimos 3 productos agregados al carrito.
13. El sistema debe dar recomendaciones de conjuntos de productos comprados juntos frecuentemente.
14. El sistema debe ser capaz de enviar notificaciones al usuario para informarle sobre ofertas y novedades para incitar al usuario a realizar nuevas compras.

2.2.1.2. No Funcionales

1. El sistema encripta las credenciales del usuario utilizando json web token
2. El sistema mantiene seguros los datos del usuario siguiendo el estándar ISO 27001
3. El sistema provee un entorno accesible siguiendo los estándares de accesibilidad W3C
4. El sistema debe guardar los datos de los usuarios en una base de datos relacional utilizando PostgreSQL

2.2.2. Supuestos

1. Asumimos que el socio formador nos entregará la información y herramientas necesarias que nos ofrece en tiempo y forma.
2. Asumimos que todos los miembros del equipo estarán presentes en las actividades de las distintas etapas del proyecto.
3. Asumimos que todos los miembros del equipo cumplirán a tiempo con las subsecciones del proyecto según el pipeline planeado.
4. Asumimos que los créditos de los servicios en la nube serán suficientes para el entrenamiento de nuestros modelos y no serán desperdiciados.
5. Asumimos que el socio formador estará presente en todas las etapas de validación.

2.2.3. Restricciones

1. Que Oracle no nos dé la información necesaria para poder iniciar el proyecto.
2. El socio formador no esté presente en las etapas de validación.
3. Que los miembros del equipo no asistan a las sesiones de trabajo.
4. Perder miembros del equipo y quedar desbalanceados en conocimientos.

2.3. Riesgos y contingencias

Riesgos	Impacto	Contingencias
Oracle nos da la base de datos muy tarde en el desarrollo.	No se entrenan los modelos a tiempo y se entrega un producto de menor calidad.	Utilizar una base de datos externa/provisional para el desarrollo del modelo.
Oracle no asiste a las reuniones de seguimiento / no responde.	No saber qué necesitan y no se tenga un producto de acuerdo a sus expectativas.	Completar el proyecto sin socio formador con apoyo de profesores coordinadores.
Los modelos no alcanzan el desempeño esperado.	No realiza bien su trabajo y la implementación es mala.	Simplificar el producto final.
La aplicación no cumple con las funcionalidades marcadas en un deadline marcado.	No se entrega el proyecto a tiempo y se ven afectadas las calificaciones.	Reducir la funcionalidad del modelo hasta donde se tiene elaborado y buscar implementar las que no cuesten tanto tiempo del proyecto.
Debido a las necesidades del proyecto, los créditos de las instancias de Oracle se acaban.	No se puede entregar nada relacionado a sus herramientas y puede generar descontento.	Tener el proyecto en repositorios y ocupar otra nube donde tengamos mayor acceso. En este caso AWS tenemos acceso y podemos pedir más créditos.
Los datos proporcionados por Oracle son tantos que es complejo o muy costoso subirlos todos a una base de datos SQL.	No se tienen modelos con buenos desempeños y no llegan a cumplir los objetivos.	Podemos utilizar una muestra de los datos que vamos a utilizar, ya que la base de datos mayormente se va a utilizar para mostrar productos en el front, tomando en cuenta que el modelo debe de entrenarse con la misma muestra.
Retraso por incompatibilidad entre librerías o versiones de dependencias.	El proyecto no se completa y se ven afectadas las calificaciones.	Documentar versiones de software y entorno virtual desde el inicio. Usar Docker para mantener la consistencia entre entornos.
Problemas de seguridad o filtración de datos del usuario.	Se filtran los datos y se genera descontento con el socio formador.	Implementar protocolos de seguridad (JWT, HTTPS, ISO 27001) y chequeos de seguridad antes del despliegue.
Fallas de conexión a servicios externos (APIs, bases de datos o nube).	El objetivo principal del proyecto se ve vulnerado y no se logra el proyecto.	Implementar mecanismos de reconexión automática y mensajes de error amigables para el usuario.

Sobrecarga de trabajo o desbalance de actividades.	El equipo se ve separado y se generan problemas internos que afectan el proyecto.	Redistribuir tareas semanalmente y establecer pares de apoyo (buddy system) entre los miembros de ingeniería.
Errores en la integración entre frontend, backend y modelo de IA.	No se tiene una aplicación presentable al finalizar el proyecto.	Establecer pruebas unitarias y de integración automatizadas antes de los despliegues; revisión cruzada de código entre integrantes.

2.4. Terminología

- **Tasa de conversión:** Es una métrica de marketing que mide el porcentaje de usuarios que completan una acción deseada, como una compra, un registro o una descarga, de un total de visitantes o interacciones. **Validar con socio formador.**
- **ticket promedio:** El valor promedio de compra que realiza cada cliente en una transacción.
- **Carrito de compras:** Sección de la tienda en línea donde los usuarios agregan productos antes de realizar el pago.
- **Chatbot / Agente conversacional:** Programa impulsado por IA que interactúa con el usuario mediante lenguaje natural para resolver dudas o asistir en el proceso de compra.
- **Tasa de abandono:** Porcentaje de usuarios que añaden productos al carrito pero no completan la compra. Es un indicador clave de fricción en la experiencia del usuario.
- **Recomendación personalizada:** Sugerencia de productos generada por un modelo de IA del catálogo de productos, basada en el historial, preferencias o comportamiento del usuario.

2.5. Costos y Beneficios

2.5.1. Costos

- Personal
 - Ivan Ramos - Ingeniero en tecnologías computacionales → Sueldo base 25,000 → Seguro social, vales de despensa y viáticos, 12 días de vacaciones pagadas al año.
 - Yose Sotomayor - Ingeniero en ciencia de datos y matemáticas | ML Engineer → Sueldo base 35,000 MXN → Prestaciones de ley + 30 días aguinaldo, 16 días de vacaciones, 30% prima vacacional, 15% vales de despensa, SGMM.
 - Carlos Zamudio - Ingeniero en tecnologías computacionales | Software Engineer → sueldo base 2,000 USD → Prestaciones de ley + 30 días aguinaldo, 16 días de vacaciones, 30% prima vacacional, 15% vales de despensa, SGMM.
 - José Antonio - Ingeniero en tecnologías computacionales | Software Engineer → sueldo base 2,000 USD → Prestaciones de ley + 30 días aguinaldo, 16 días de vacaciones, 30% prima vacacional, 15% vales de despensa, SGMM.

- Ameyalli Contreras - Ingeniera en ciencia de datos y matemáticas → Sueldo base 25,000 MXN → Prestaciones de ley, 12 días de vacaciones, 30% de prima vacacional, 15% vales de despensa, SGMM.
- Daniel Bernal - Ingeniero en robótica y sistemas digitales → Sueldo base 2,500 USD → Prestaciones de ley + 30 días aguinaldo, 16 días de vacaciones, 30% prima vacacional, 15% vales de despensa, SGMM.
- Alan Contreras - Ingeniero en ciencia de datos y matemáticas → Sueldo base 30,000 MXN → Prestaciones de ley, 12 días de vacaciones, 30% de prima vacacional, 15% vales de despensa, SGMM.
- Alan Vega - Ingeniero en tecnologías computacionales → Sueldo base: 35,000 MXN → Prestaciones de ley, 12 días de vacaciones, 30% de prima vacacional, 15% vales de despensa, SGMM
- Oficinas (Lugar de trabajo): Co-work Polanco, 2000 x persona (8). 16,000 pesos al mes.
- Créditos de Oracle Cloud Infrastructure (300 USD por integrante del equipo).
- Tiempo a invertir en el proyecto: 11 semanas.

2.5.1.1. Costo total:

- Aproximadamente 296,000 MXN al mes: 3,256,000 en 11 semanas
- 48000 MXN de créditos de Oracle Cloud Infrastructure

2.5.2. Beneficios

- Aumento de ingresos del 10% (por incremento en ventas totales).
 - Aumento de ingresos del 7% (por interacciones de IA y reconocimiento de imágenes).
 - Aumento de ingresos del 3% (por notificaciones y recomendaciones de descuentos).
- Aumento de la tasa de conversión de carrito del 10%.
- Disminución de clics en el proceso de búsqueda del 20%.
- Disminución de la tasa de abandono de usuarios del 10% (de usuarios inactivos).
- Aumento de la tasa de retención de clientes del 5% (por mayor satisfacción y menos abandono).
- Recuperación de clientes inactivos del 3%.

3. Determinación de objetivos de minería de datos

Objetivo de Negocio	3.1 Objetivos de Minería de Datos	3.2. Criterios de Éxito
NP.001	Regresión: Predecir la probabilidad de que un cliente adquiera un producto recomendado por el modelo como producto relacionado al producto que está observando el cliente. Se considera la regresión como lo puede ser la logística, ya que la probabilidad es binaria, considerando si el cliente lo compra o no lo compra.	Precision > 80% para asegurar que el modelo muestra productos que los clientes encuentran atractivos.
NP.001.1	Clustering: Identificar grupos y tipos de productos que permitan que el agente de IA provea productos relacionados a las preferencias de los usuarios. Se aplica con el fin de descubrir los grupos naturales orientados a los productos.	Encontrar por lo menos un cluster interpretable por tipo de producto definido que contenga > 60% de los productos de la categoría de producto
NP.001.2	Regresión: Identificar la probabilidad de que un usuario le haga caso a una notificación de producto y termine por comprar el producto Clasificación: Clasificar a los usuarios entre usuarios que ponen atención a las notificaciones y compraron productos y usuarios que no. Se definen estas herramientas ya que se evalúa el atender la notificación.	Precision > 80% en la predicción de si el usuario hará caso a una notificación Recall > 80% en la clasificación de usuarios que si hacen caso a las notificaciones
NP.002	Encontrar: Esto con la finalidad de identificar patrones de compra y productos frecuentemente adquiridos juntos para recomendar combinaciones que aumenten el número de productos añadidos al carrito.	Lograr que las recomendaciones basadas en reglas de asociación incrementen en al menos 5% el número promedio de artículos por carrito dentro de las primeras 6 semanas del despliegue.

NP.003	Clasificación: Clasificar a los usuarios dependiendo de si están satisfechos con la página web o no Clustering: Identificar grupos de usuarios insatisfechos y revisar sus quejas para poder identificar problemas comunes a resolver para mejorar la satisfacción. Se realiza la agrupación para después realizar la clasificación y así tener mejores resultados orientados al objetivo.	Lograr detectar correctamente al menos el 70% de los usuarios insatisfechos reales, permitiendo actuar sobre ellos y mejorar la satisfacción general de la página en al menos un 10% tras las mejoras implementadas.
NP.003.1	Clasificación: Predecir la categoría o subcategoría del producto más probable que el usuario desea, a partir del texto introducido en el chat. Obteniendo una reducción de clics promedio hasta 0 clics en la búsqueda y selección del producto.	Conseguir una precisión de al menos 85% en la predicción de la categoría correcta.
NP.004	• Identificar factores que influyen en la inactividad del usuario, como tiempo promedio en el sitio, frecuencia de compra o interacción con campañas. • Predicción: Predecir la probabilidad de abandono o inactividad del usuario utilizando modelos de clasificación (como regresión logística, random forest o redes neuronales). • Clustering: Segmentar usuarios según su comportamiento y nivel de riesgo de abandono para aplicar campañas de retención específicas.	• Detectar al menos 3 variables clave asociadas a la inactividad y diseñar estrategias personalizadas que disminuyan el abandono en un 10% en el primer trimestre. • Alcanzar una precisión de al menos 85% en la predicción de usuarios inactivos y reducir en 5% la tasa de inactividad mensual dentro de los primeros 2 meses de implementación. • Conseguir que las campañas dirigidas a segmentos de alto riesgo logren reactivar al menos 30% de usuarios inactivos en un periodo de 6 semanas.

4. Producción del plan del proyecto

4.1. Plan del proyecto

En el siguiente archivo se encuentra la distribución de responsabilidades y actividades del equipo:

 Equipo2_Plan del Proyecto.xlsx

El cronograma tiene una duración aproximada de seis semanas y está estructurado por secciones del proyecto: *Frontend*, *Backend*, *Agente*, *Base de Datos*, *Modelación de IA e Implementación en la Nube*. Cada sección se desglosa en las tareas necesarias para considerarla completada, indicando la fase correspondiente del modelo **CRISP-DM**, el responsable asignado, así como las fechas de inicio y entrega. En la parte derecha del documento se presenta una visualización temporal que muestra la duración estimada de cada tarea a lo largo del proyecto.

4.2. Evaluación inicial de herramientas y técnicas

4.2.1. Lista de herramientas

Herramienta	Justificación
Python (FastAPI)	Framework moderno y de alto rendimiento utilizado para construir la Capa de Servicios (<i>Backend</i>). Gestiona las Endpoints y actúa como el puente principal entre el <i>Frontend</i> , la Base de Datos y los módulos de IA.
ML Flow	Plataforma para la gestión del ciclo de vida de ML. Permite el seguimiento, versionamiento y despliegue de los modelos entrenados por PySpark, facilitando el proceso de Reentrenamientos (Sync).
PySpark	Framework escalable de procesamiento de datos utilizado en el módulo <i>ML Flow</i> para el entrenamiento y reentrenamiento de modelos de Machine Learning (ej. recomendación, clustering).
PySpark Core (pyspark.sql)	Es el framework principal. Se utiliza para la carga inicial de datos, la manipulación de DataFrames a gran escala y las tareas básicas de limpieza y Análisis Exploratorio de Datos (EDA).
PySpark MLlib (pyspark.ml.feature)	Contiene todas las transformaciones necesarias para el preprocesamiento, como <i>VectorAssembler</i> (para combinar características), <i>StringIndexer</i> y escaladores (<i>StandardScaler</i>), que preparan los datos para los algoritmos.
PySpark MLlib (pyspark.ml.clustering)	Módulo que proporciona los algoritmos de clustering escalables, como <i>KMeans</i> o <i>BisectingKMeans</i> , que se usarán para entrenar el modelo y encontrar las agrupaciones dentro de los datos.
PySpark MLlib (pyspark.ml.evaluation)	Necesario para evaluar el rendimiento de los modelos. Contiene el <i>ClusteringEvaluator</i> para calcular métricas como el Coeficiente de Silueta, fundamental para seleccionar el número óptimo de clusters (<i>k</i>).
PySpark MLlib (pyspark.ml)	Esencial para la creación del Pipeline, que encadena de forma organizada todos los pasos de preprocesamiento y modelado. También se usa para guardar y cargar el modelo entrenado.

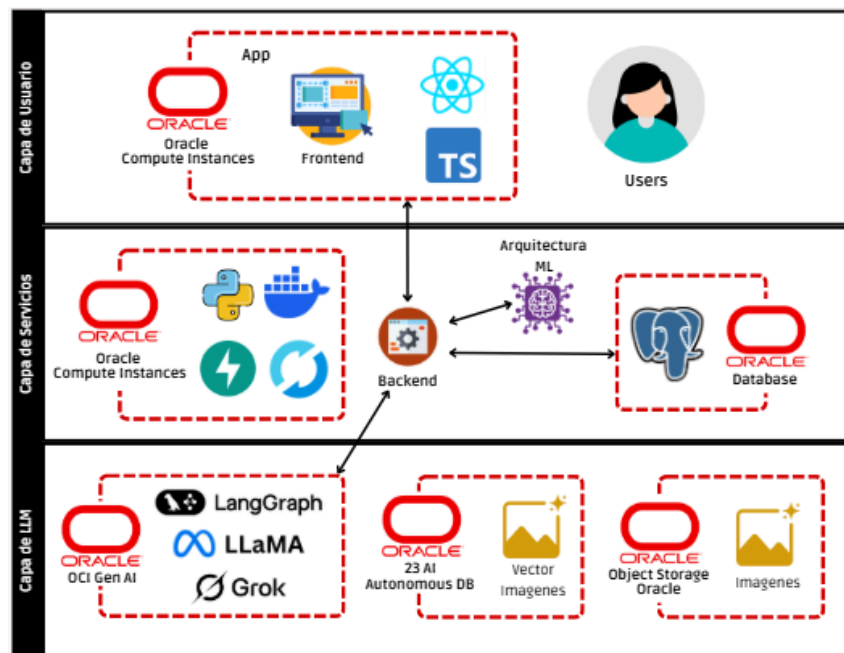
Spark JDBC (Java Database Connectivity)	Método que <i>PySpark</i> utiliza para conectarse a bases de datos relacionales externas como <i>PostgreSQL</i> . Se usa específicamente para exportar el DataFrame final (con las asignaciones de cluster) a tu base de datos.
Driver de PostgreSQL (Archivo JAR)	Es el archivo binario (controlador) que debe proporcionarse a <i>PySpark</i> al iniciar la sesión. Permite la comunicación real y física (el protocolo) entre el entorno <i>Spark</i> y la base de datos <i>PostgreSQL</i> .
PostgreSQL (Database/DB)	Base de datos relacional para el almacenamiento primario de los datos transaccionales de la tienda y como fuente de datos para el motor RAG.
Chroma (ChromaDB)	Base de datos de vectores esencial para la funcionalidad RAG (<i>Retrieval-Augmented Generation</i>), permitiendo al agente de IA buscar y recuperar información contextualmente relevante.
LangChain	Framework de orquestación para construir y gestionar la lógica de los Agentes (como <i>Kimi 2</i>), conectando el LLM con las herramientas y las fuentes de datos (<i>ChromaDB</i> , bases de datos).
Ollama / LLM (Large Language Model)	Proporciona el motor de lenguaje natural para que el agente pueda interpretar las consultas del usuario y generar respuestas coherentes.
CLIP / Módulo de Imágenes	Utilizado para el procesamiento y vectorización de imágenes. Genera los embeddings necesarios para la funcionalidad de búsqueda de productos por reconocimiento de imágenes.
S3 (Amazon Simple Storage Service)	Servicio de almacenamiento de objetos escalables para guardar archivos grandes y persistentes, como la base de imágenes de productos y datos de entrenamiento de Initialization.
React + TypeScript (App/Frontend)	Tecnologías utilizadas para desarrollar la Capa de Usuario (<i>Frontend</i>), proporcionando una interfaz dinámica y tipada para que los usuarios interactúen con la tienda y el chatbot.

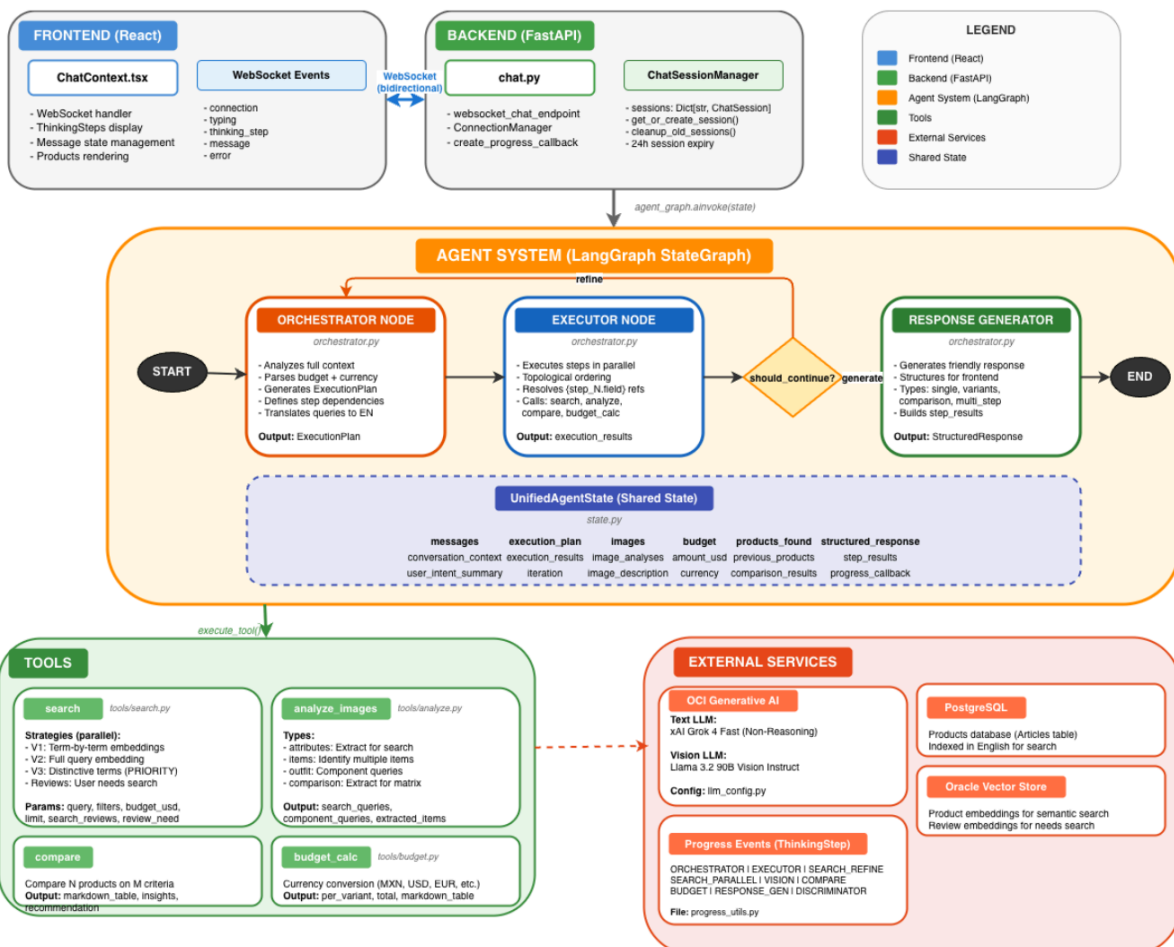
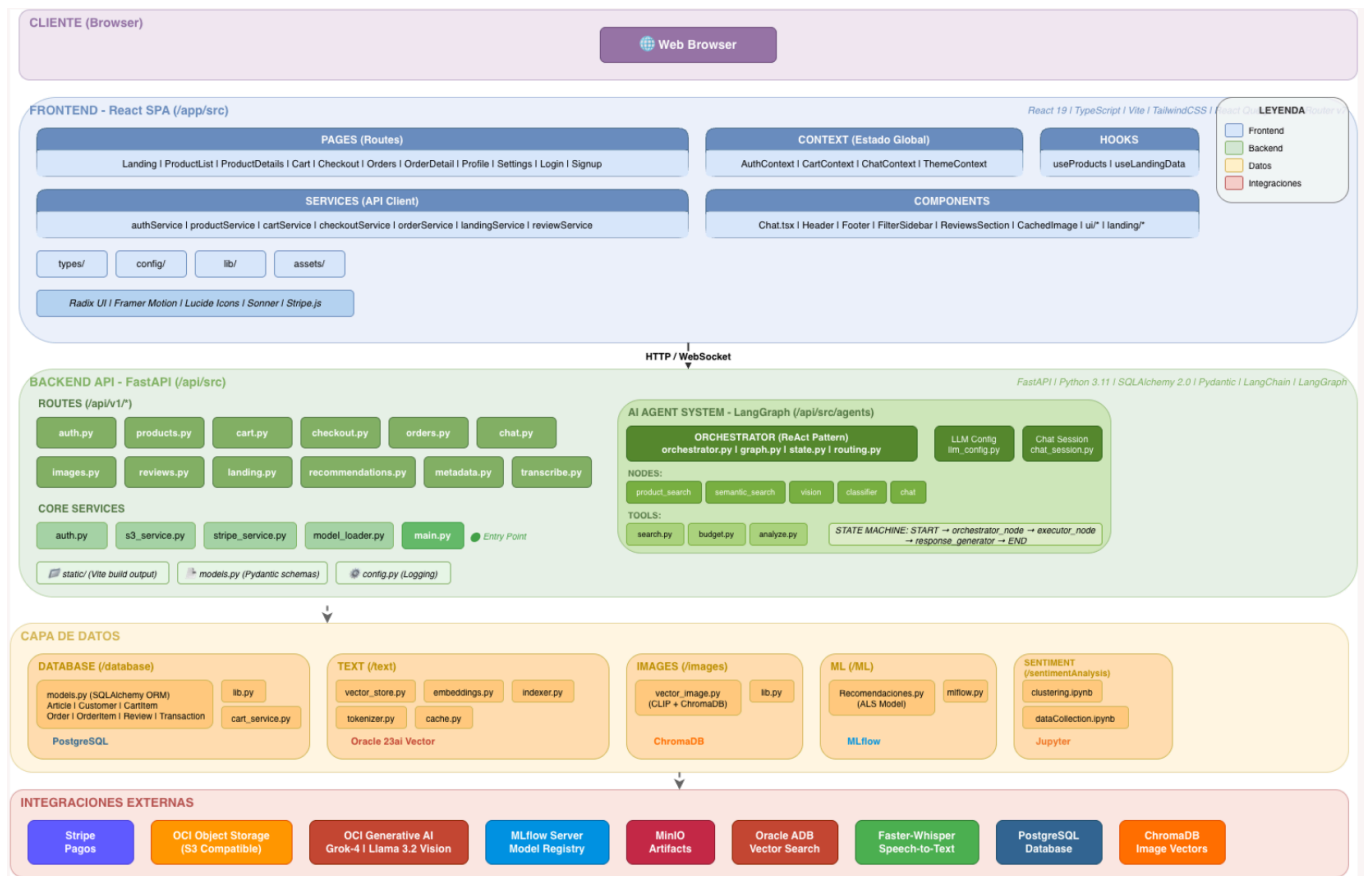
4.2.2. Lista de técnicas (provisional)

Técnicas	Justificación
K-Means (Clustering)	Algoritmo simple, rápido y altamente escalable con PySpark, ideal para segmentar a los usuarios de la tienda de retail basándose en sus patrones de compra. Así como, para segmentar los productos en categorías variadas.
Random Forest Classifier (Clasificación)	Algoritmo de conjunto (ensemble) robusto, ideal para obtener alta precisión en la predicción de la propensión de compra. Maneja bien las características no lineales y reduce el sobreajuste (overfitting).
Collaborative filter (Sistemas de Recomendación)	Se justifica porque usa las preferencias de muchos usuarios para recomendar productos a otros con gustos similares, mejorando la personalización.

Market basket analysis (Sistemas de Recomendación)	Se justifica porque identifica productos que suelen comprarse juntos, ayudando a impulsar ventas cruzadas y estrategias de recomendación basadas en asociaciones.
Matching Semántico (Sistema de recomendación, Review Analysis)	Aprovecha el contenido real de las reseñas para hacer “matching” semántico entre lo que el usuario pide y lo que otros compradores describen. Usa vectores semánticos (representaciones numéricas del significado) para encontrar reseñas relevantes, lectura conjunta del par pedido–reseña para afinar la relevancia, inferencia natural del lenguaje para detectar si una reseña apoya o contradice requisitos.

4.2.3. Diagramas de arquitectura final





INFRAESTRUCTURA



EDA H&M.ipynb

6. Selección de Datos

6.1. Tabla Articles

- *article_id*: Es el identificador único del producto, indispensable para vincular las transacciones con los artículos. En el contexto de recomendaciones personalizadas y predicciones de compra, es la clave para mapear comportamientos de usuarios con artículos concretos.
- *prod_name*: Aporta una descripción legible del producto que puede usarse para mostrar resultados en interfaces o para generar embeddings textuales en el modelo de recomendación (content-based). Mejora la “facilidad de encontrar lo que buscas”.
- *product_type_name*: Describe el tipo de prenda o accesorio (camisa, falda, abrigo, etc.), lo que facilita agrupar productos por similitud visual o funcional. Es esencial para sistemas de recomendación basados en contenido y para entender preferencias de estilo en clientes individuales o clústeres.
- *product_group_name*: Indica la categoría general del producto (ej. Blouses, Shoes). Es relevante para entender patrones de compra y segmentar preferencias, permitiendo sugerir productos del mismo grupo o analizar fidelidad hacia ciertos tipos de prendas.

- *garment_group_name*: Ofrece una clasificación más específica dentro del grupo (por ejemplo, Nightwear dentro de Underwear). Es útil para granularidad en recomendaciones y análisis de afinidad fina entre productos.
- *colour_group_name*: Las preferencias de color son un componente estético fuerte en decisiones de compra. Incluirlo mejora la personalización de recomendaciones, y puede revelar patrones de abandono o fidelidad por tonos específicos.
- *graphical_appearance_name*: Aporta información clave sobre el patrón visual y estilo del producto, lo que influye directamente en las preferencias estéticas del cliente y en su decisión de compra. Es útil para recomendaciones personalizadas, predicción de compra y mejora de la experiencia de búsqueda visual, ya que permite identificar similitudes entre prendas y adaptar la oferta a los gustos individuales sin generar redundancia ni exceso de categorías.
- *department_name*: Nivel menos granulado pero igualmente detallado del tipo de producto (por ejemplo, Jersey Basic, Clean Lingerie, etc). Es útil en modelos de predicción y clustering, pues puede capturar diferencias generales pero no tan granulares en estilo o tipo de prenda que no aparecen en los índices superiores. También es informativa para el diseño de estrategias de surtido o promociones específicas.
- *index_group_name*: Nivel de agrupación superior del tipo de producto (por ejemplo, Menswear, Ladieswear, Sport). Es una variable categórica clave para segmentar preferencias generales de clientes, mejorar la relevancia de recomendaciones y facilitar análisis de mercado o clústeres por tipo de producto. Además, sirve para interpretar la composición de la oferta (por departamento o segmento).
- *index_name*: Subcategoría más específica que complementa *index_group_name*. Esta granularidad ayuda en la personalización de recomendaciones (por ejemplo, un cliente interesado en Ladieswear - Lingerie recibirá sugerencias dentro de la misma subcategoría). También es valiosa en modelos de predicción de compra, ya que refleja patrones de preferencia dentro de una categoría amplia.
- *section_name*: Ofrece un nivel de detalle ideal para los modelos de recomendación y predicción, pues, al capturar simultáneamente el público objetivo (como mujeres, hombres, niños o bebés), el tipo de producto (por ejemplo, básicos, accesorios o ropa deportiva) y la línea comercial (como Premium, Trend o Collaborations). Esta combinación de información semántica permite entender no sólo qué tipo de prendas se venden, sino en qué contexto y para qué cliente, lo que la hace sumamente útil para la segmentación de clientes, la personalización de recomendaciones y la predicción de comportamiento de compra. Además, equilibra bien la riqueza descriptiva sin generar un exceso de categorías, manteniendo interpretabilidad y valor analítico para los modelos de negocio de la tienda.
- *detail_desc*: Texto descriptivo del artículo; muy útil para generar representaciones semánticas. Mejora la capacidad del sistema para entender similitud conceptual entre productos.
- *product_code*, *colour_group_code*, *garment_group_no*: Se considera conservarlos en caso de que los modelos requieran codificación numérica o se usen como

identificadores alternativos. Son útiles para reducir la cardinalidad en variables categóricas muy largas durante el entrenamiento.

Columnas a descartar:

- *product_type_no*, *graphical_appearance_no*, *perceived_colour_value_id*, *department_no*, *section_no*, *index_code*, *index_group_no*, *perceived_colour_master_id*: Son códigos numéricos redundantes, es decir, representan lo mismo que las columnas categóricas legibles (como *product_group_name*, *colour_group_name*, etc.). No aportan valor directo al modelo ni a la interpretación del negocio.
- *perceived_colour_value_name*, *perceived_colour_master_name*: Aunque son descriptivas, tienen alta relación con *colour_group_name* y agregan redundancia sin mejorar la precisión de forma significativa.

6.2. Tabla Customers

Se decidió conservar en su totalidad las columnas contenidas en esta tabla. A continuación se muestra la justificación para la conservación de cada una de ellas:

- *customer_id*: Identificador único, esencial para unir con transacciones. Es la base de cualquier modelo personalizado o de predicción de abandono.
- *age*: Variable demográfica crítica para segmentar preferencias de estilo y gasto. Permite que el sistema recomiende productos adecuados a segmentos generacionales y detectar cambios en comportamiento de compra con la edad.
- *club_member_status*: Indica el nivel de fidelidad o membresía, útil para el modelo de abandono (clientes “ACTIVE” tienden a permanecer) y para segmentación (clientes premium vs. nuevos).
- *fashion_news_frequency*: Refleja el nivel de interacción con comunicaciones de la marca (suscripción a newsletters, notificaciones). Es una señal directa de engagement y predictor del comportamiento de compra o abandono.
- *postal_code*: Permite agrupar clientes por región o perfil socioeconómico, útil para recomendaciones por disponibilidad regional o análisis de clusters geográficos. A pesar de que los códigos se encuentran codificados, estos mantienen coherencia interna (es decir, el mismo cliente conserva el mismo código en todas sus transacciones), por lo que, funciona como un identificador de región anónima para medir lealtad o consistencia de comportamiento local a pesar de no tener acceso a una ubicación geográfica exacta.
- *FN (fashion news)* y *Active (actividad)*: Indicadores binarios de engagement pasivo y activo, ideales como features en modelos de abandono o compra: quienes no interactúan o no están activos suelen tener mayor riesgo de churn.

6.3. Tabla Transactions

Al igual que en la tabla anterior, se decidió conservar la totalidad de columnas que contiene esta tabla. A continuación se muestra la justificación por cada una de las columnas:

- *t_dat*: Fecha de la transacción; clave para todos los modelos temporales: frecuencia de compra, recencia (última compra), y análisis de abandono. Permite crear variables de comportamiento.
- *customer_id*: Une el comportamiento transaccional con características del cliente (*customers.csv*), esencial para modelos de recomendación, compra y abandono.
- *article_id*: Relaciona cada compra con la descripción del producto (*articles.csv*), habilitando el entrenamiento de recomendadores basados en interacciones.
- *price*: Refleja la sensibilidad al precio y el valor del producto adquirido. Es útil para predecir futuras compras (compradores de alto o bajo ticket) y segmentar clientes por poder adquisitivo.
- *sales_channel_id*: Identifica si la compra fue online o en tienda física; es una señal conductual que permite ajustar estrategias de recomendación y predecir abandono (clientes que compren solo en un canal tienden a menos interacciones si ese canal cambia). Además servirá para la creación de variables más adelante en el presente reporte.

6.4. Final Shapes

Python

```

===== ARTICLES =====
> Before: (105542, 25)
> After:  (105542, 15)

===== CUSTOMERS =====
> Before: (1371980, 7)
> After:  (1371980, 7)

===== TRANSACTIONS =====
> Before: (31788324, 5)
> After:  (31788324, 5)

```

7. Limpieza de Datos

7.1. Valores faltantes

7.1.1. Tabla Articles

En este dataset el único campo que presenta valores faltantes es la columna *detail_desc* que corresponde a la descripción del producto, sin embargo, se observó que algunos de los productos que no cuentan con una descripción son las variantes de un mismo producto. Por ejemplo: Si el producto *JERSEY WINTER* tiene variantes en colores negro, azul y rojo, únicamente uno de estos 3 contiene la descripción. Por ello se procedió a copiar la descripción del producto en sus diversas variantes.

A continuación se muestra la reducción de valores faltantes tras la imputación anteriormente mencionada:

```
Python
> Faltantes antes de la imputación:
  detail_desc NaN:  416

> Faltantes después de la imputación:
  detail_desc NaN:  356
```

Para realizar la imputación de las descripciones faltantes en la tabla `articles`, se aplicó una estrategia de propagación basada en el nombre del producto (*prod_name*). Este método permite asignar la misma descripción a todas las variantes de un mismo artículo (por ejemplo, diferentes colores), garantizando coherencia y reduciendo la cantidad de valores nulos de forma semánticamente válida.

Pese a ello, 356 productos (0.34% aproximadamente) permanecen sin descripción, debido a que ninguna de sus variantes contaba con información en el campo *detail_desc* desde la fuente original. En consecuencia, no existen datos de referencia que permitan una imputación coherente.

Se decidió conservar estos valores como nulos para mantener la integridad del dataset y evitar generar descripciones artificiales o potencialmente erróneas. Estos casos serán reservados para una imputación posterior mediante generación de texto automática en caso de ser indispensables para los objetivos y necesidades futuras de la sección de modelado.

7.1.2. Tabla Customers

En este dataset, se realizarán tres procesos distintos para el llenado de los valores faltantes.

1. **Variables Booleanas:** Reemplazar NaN por 0.
 - *Active*: Esta variable representa si el cliente está activo en el programa de membresía o cuenta en línea. Los NaN sugieren que el usuario no se considera activo o no ha interactuado recientemente, por lo tanto imputar 0 refuerza esa interpretación. Esta codificación facilita el análisis de retención, predicción de abandono y correlación con frecuencia de compra, manteniendo la variable en formato binario interpretable.
 - *FN*: Esta variable indica si el cliente tiene un número de fidelización o una cuenta formal registrada. Los valores faltantes implican ausencia de esa condición, por lo que asignar 0 (no tiene FN) conserva el significado lógico y permite al modelo distinguir entre clientes registrados y visitantes, información clave para segmentación y predicción de compra.

Resultados de la imputación:

Python

```
Active NaN: 0
> Uniques: [0. 1.]

FN NaN: 0
> Uniques: [0. 1.]
```

2. **Variables Multietiqueta:**

- *fashion_news_frequency*: Reemplazar los NaN por 'NONE'.

Los clientes que no reciben el boletín de moda probablemente nunca se suscribieron o rechazaron las notificaciones. Imputar con 'NONE' preserva el significado lógico de “sin frecuencia de comunicación” y evita introducir sesgos que sobreestimen el contacto con el cliente. Además, esta decisión mantiene coherencia con los propósitos de negocio, ya que permite distinguir entre clientes activos en marketing (suscritos) y pasivos (no interesados), algo relevante para segmentación, predicción de compra y abandono.

- *club_member_status*: Reemplazar los NaN por 'NOT-MEMBER'.

Esta variable indica el estado del cliente dentro del programa de membresía, distinguiendo entre miembros activos, en proceso de registro y exmiembros. Los valores faltantes corresponden a clientes que no cuentan con ningún registro en el club, es decir, compradores sin afiliación. Por ello, imputar los NaN con 'NOT-MEMBER' mantiene la lógica de negocio y evita confundir la ausencia de registro con inactividad. Esta imputación conserva información útil para los modelos de recomendación y predicción de abandono, al permitir identificar el comportamiento diferenciado entre miembros y no miembros.

Resultados de la imputación:

Python

```
fashion_news_frequency NaN: 0
> Uniques: ['NONE' 'Regularly' 'Monthly']

club_member_status NaN: 0
> Uniques: ['ACTIVE' 'NOT-MEMBER' 'PRE-CREATE' 'LEFT CLUB']
```

3. **Variable Numérica:** Imputar los NaN utilizando KNN-Imputer.

- *age*: La edad está directamente relacionada con otras características del cliente, como su tipo de membresía, nivel de actividad, y frecuencia de interacción. Por ello, un método contextual o basado en vecinos similares es más adecuado que el uso de métricas estadísticas como el promedio global o la mediana, ya que, conserva las correlaciones naturales del dataset.

Resultados de la imputación:

```
Python
> Faltantes antes de la imputación
  age NaN:  15,861
> Faltantes después de la imputación
  age NaN:   0
```

7.2. Valores atípicos

Los únicos outliers que se encontró conveniente tratar son los presentes dentro de la columna *age* de la tabla *Customers* ya que, es el único valor numérico que en este caso no representa un ID o categoría de artículo (como ocurre en los outliers de la tabla *Articles*).

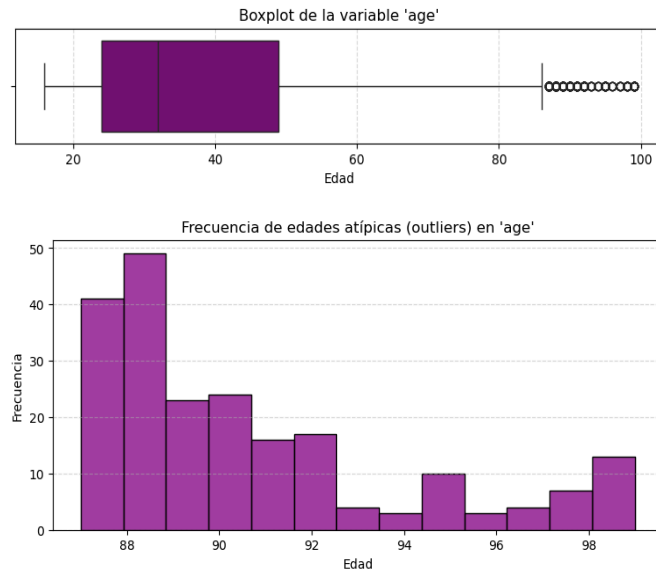
La variable *age* representa la edad declarada o estimada de cada cliente. Este atributo es relevante para comprender los patrones de compra, segmentar audiencias y ajustar estrategias de recomendación personalizadas. Tras un análisis exploratorio, se identificó la presencia de clientes menores de edad (16–17 años) y de valores atípicos superiores (mayores de 87 años).

El segmento de usuarios en el rango de entre 16 y 17 años no representa un error, sino un reflejo realista del público objetivo de la marca, que incluye adolescentes interesados en la moda y el comercio electrónico. En la práctica, las políticas de muchas tiendas en línea permiten el registro de usuarios de 16 años o más, siempre que las compras se realicen bajo supervisión o consentimiento parental. Además, este grupo aporta información valiosa sobre un segmento emergente con alto potencial de fidelización a largo plazo y patrones de consumo distintivos. Por lo tanto, se conservarán estos registros para los modelos de segmentación y recomendación, considerando que su participación en predicciones de compra directa podrá ser analizada por separado si las políticas del negocio lo requieren.

A continuación se muestra el resultado y conteo de outliers para la variable *age*:

```
Python
Total de outliers en la columna 'age': 214

Porcentaje sobre el total: 0.02%
Edad mínima entre outliers: 87.0
Edad máxima entre outliers: 99.0
```



Los gráficos evidencian que los valores atípicos en la variable *age* se concentran principalmente entre los 87 y 99 años, con mayor densidad en el rango de 87–92 años. Aunque estas edades son poco comunes dentro de una tienda, no necesariamente representan errores, pues son posibles, pero sí constituyen valores extremos respecto a la distribución general del conjunto de clientes.

Los outliers moderados (87–92 años) muestran una frecuencia relativamente alta, lo que sugiere que corresponden a observaciones válidas dentro de un segmento minoritario de adultos mayores. En cambio, los valores superiores a 93 años se presentan con frecuencia mínima y comportamiento disperso, lo que indica que probablemente son errores de digitación, valores aproximados (como 99), o registros hechos por otras personas (un familiar creando la cuenta a una persona de la tercera edad, en cuyo caso el adulto mayor no sería el comprador principal). Además, en el análisis del grupo de clientes con edades registradas entre 93 y 99 años, se observó que la mayoría realizó muy pocas compras, mientras que unos pocos registros presentaron actividad inusualmente alta. Para manejar esta situación sin perder información valiosa, se decidió aplicar un *capping*, ajustando todas las edades mayores a 92 años a un máximo de 92. Esta estrategia permite conservar los 44 clientes con valores extremos, minimizando el impacto de outliers en métricas y modelos predictivos, y reflejando de manera más realista la actividad de compra del grupo de edad avanzada.

Python

Edad máxima registrada: 92.0

7.3. Ruido e inconsistencias

Dentro del análisis de inconsistencias y ruido, únicamente se encontró necesario tratar los duplicados, presentes específicamente en la tabla de transacciones.

Para tratar los registros duplicados dentro del conjunto de transacciones, se aplicó un proceso de capping con el objetivo de conservar la coherencia de los datos sin eliminar información relevante. Dado que las transacciones representan compras individuales de productos, es posible que existan múltiples registros idénticos cuando un cliente adquiere varias unidades del mismo artículo en una misma fecha. Sin embargo, se identificaron casos con repeticiones excesivas (hasta 569 veces), que representan comportamientos atípicos o posibles errores de captura. Por ello, se estableció un límite máximo de repeticiones por combinación de cliente, producto y fecha (hasta 30 compras idénticas), manteniendo únicamente las primeras ocurrencias dentro de ese umbral y eliminando el exceso. Esta estrategia reduce el impacto de valores extremos, evita distorsiones en los análisis posteriores y preserva la representatividad general del comportamiento de compra real.

Python

```
Transacciones originales: 31,788,324  
Transacciones después de capping: 31,784,677  
Registros eliminados por capping: 3,647
```

8. Construcción de Datos

8.1. Creación de Features

8.1.1. Precio por artículo

Debido a que el catálogo de artículos no cuenta con columna de precios, la cual es indispensable tanto para el desarrollo de la página, como para un buen modelado, se tomaron los precios provenientes del conjunto de datos de transacciones, donde inicialmente se ajustaron multiplicando el valor original de price por 590, con el fin de convertirlo a una escala monetaria coherente con el valor real de venta de los artículos.

Python

```
> Mínimo precio real: 0.01  
> Máximo precio real: 349.0
```

Posteriormente, se realizó un proceso de selección por artículo y canal de venta (*sales_channel_id*), priorizando los registros correspondientes al canal 2 (ventas en línea) para capturar los precios representativos del entorno digital. En los casos donde no existían ventas en dicho canal, se tomó el precio del canal 1 (ventas en tienda física) como respaldo. Una vez determinado el precio más representativo para cada *article_id*, estos valores fueron fusionados con la tabla *articles*, integrando la información de precios al catálogo principal de productos.

Este proceso de imputación de precios se llevó a cabo con el objetivo de preservar la coherencia económica y semántica dentro del catálogo de productos. En primer lugar, se priorizó el uso de la moda (valor más frecuente) como estimador del precio representativo, lo cual permite reflejar el valor más estable del artículo y mitigar el impacto de descuentos temporales o errores de registro. Asimismo, se dio prioridad al canal 2 (ventas en línea), en coherencia con el objetivo del análisis enfocado en el comportamiento digital de los clientes. En ausencia de datos en dicho canal, se recurrió a la información del canal 1 (tiendas físicas), asegurando cobertura completa y evitando dejar artículos sin valor económico asignado. De esta forma al pasar los precios a la tabla *articles*, se consiguió preservar la relación entre productos y sus atributos originales como tipo y grupo o categoría, lo que puede facilitar análisis posteriores de precios por segmento.

Python

```
> Artículos sin precio asignado: 995
```

Adicionalmente, para los artículos que no registraron ventas en ningún canal que por consiguiente, quedaron sin precio asignado en el catálogo, se aplicó una imputación basada en la mediana del tipo de producto (*product_type_name*), garantizando consistencia interna dentro de cada categoría. Esta estrategia mantiene proporciones realistas entre prendas similares y previene distorsiones en modelos de recomendación o predicción de demanda.

Python

```
> Artículos sin precio asignado tras imputación con mediana: 1
```

En los pocos casos donde un tipo de producto carecía completamente de registros, se utilizó la mediana de una categoría superior *product_group_name*, asegurando la completitud de la base y la integridad analítica del conjunto de datos.

Python

```
> Artículos sin precio tras imputación completa: 0
```

8.1.2. Variables temporales

Las variables temporales fueron generadas a partir del historial de transacciones, utilizando las fechas de compra registradas en el campo *t_dat* y su relación con cada cliente. A partir de ello, se calcularon indicadores clásicos de comportamiento temporal como *frequency* (número total de compras realizadas), *monetary_value* (suma del gasto total), *recency* (días transcurridos desde la última compra hasta la fecha de referencia) y

days_since_last_purchase (diferencia temporal entre la compra más reciente y el último registro del cliente).

Estas variables permiten capturar la dinámica del comportamiento de compra de los clientes en el tiempo. *frequency* y *monetary_value* cuantifican la intensidad y valor económico de las compras, proporcionando una medida directa del nivel de compromiso del cliente con la tienda. *recency* y *days_since_last_purchase* reflejan la actualidad de la relación del cliente con la marca, siendo indicadores clave en modelos de predicción de retención o propensión a compra. En conjunto, estas variables permiten identificar patrones temporales de fidelización y segmentar clientes según su ciclo de vida comercial, lo que fortalece la capacidad del modelo para anticipar comportamientos futuros.

Resultado de la creación de variables temporales:

```
Python
#      Column                                Non-Null Count  Dtype
---  -
0      customer_id                          1371980 non-null  object
1      FN                                    1371980 non-null  float64
2      Active                               1371980 non-null  float64
3      club_member_status                    1371980 non-null  object
4      fashion_news_frequency                1371980 non-null  object
5      age                                    1371980 non-null  float64
6      postal_code                           1371980 non-null  object
7      frequency                             1362281 non-null  float64
8      monetary_value                       1362281 non-null  float64
9      recency                              1362281 non-null  float64
10     days_since_last_purchase              1362281 non-null  float64
```

Para el llenado de los valores faltantes, los cuales se presentan debido a que no todos los clientes han realizado transacciones dentro de la ventana de tiempo contemplada por el dataset, se ocuparon dos valores distintos:

- *frequency* y *monetary_value*: Se llenaron los valores faltantes con 0, debido a que dentro de la ventana de tiempo estudiada no se ha reflejado ninguna compra del cliente.
- *recency* y *days_since_last_purchase*: Se llenaron los valores faltantes con la cantidad de días que cubre el periodo total de transacciones, debido a que puede que el cliente haya comprado antes del periodo registrado, sin embargo al menos tiene todo ese tiempo sin realizar una compra lo cual nos permite tener un valor extremo interpretable en vez de un NaN.

Este valor se calculó de la siguiente manera:

Python

```
total_days =  
(transactions["t_dat"].max() - transactions["t_dat"].min()).days
```

8.1.3. Variables de comportamiento

Las variables de comportamiento se derivaron mediante agregaciones sobre el historial de compras por cliente, con el objetivo de caracterizar sus patrones de consumo y preferencias. *daily_avg_spent* se calculó dividiendo el gasto total entre los días en los que el cliente realizó transacciones, permitiendo medir la intensidad económica diaria. *unique_items_bought*, *unique_type_bought* y *unique_group_bought* se generaron contando los artículos, tipos y grupos de producto distintos adquiridos por cada cliente, lo que refleja la amplitud y diversidad de sus intereses, lo que permite distinguir entre compradores especializados y clientes con hábitos de compra diversificados. Finalmente, *fav_group*, *fav_type* y *fav_color* se obtuvieron identificando la categoría, tipo y color más frecuentes en las compras de cada cliente, estas sintetizan las preferencias individuales de cada cliente, funcionando como una representación de su estilo de consumo. Este conjunto de atributos es esencial para modelos de segmentación, recomendación de productos y análisis de afinidad entre clientes.

Resultado de variables de comportamiento:

Python

#	Column	Non-Null Count	Dtype
0	customer_id	1371980 non-null	object
1	FN	1371980 non-null	float64
2	Active	1371980 non-null	float64
3	club_member_status	1371980 non-null	object
4	fashion_news_frequency	1371980 non-null	object
5	age	1371980 non-null	float64
6	postal_code	1371980 non-null	object
7	frequency	1371980 non-null	float64
8	monetary_value	1371980 non-null	float64
9	recency	1371980 non-null	float64
10	days_since_last_purchase	1371980 non-null	float64
11	daily_avg_spent	1362281 non-null	float64
12	unique_items_bought	1362281 non-null	float64
13	unique_type_bought	1362281 non-null	float64
14	unique_group_bought	1362281 non-null	float64
15	fav_group	1362281 non-null	object
16	fav_type	1362281 non-null	object
17	fav_color	1362281 non-null	object

El llenado de datos faltantes se realizó llenando con 0 para las variables *daily_avg_spent*, *unique_items_bought*, *unique_type_bought* y *unique_group_bought*. Mientras que para las variables *fav_group*, *fav_type* y *fav_color* se llenaron los NaNs con la categoría *no_purchases*.

8.1.4. Variables de producto

La variable *popularity_score* mide la frecuencia relativa de compra de cada artículo dentro del total de transacciones dividiendo el número de veces que un producto fue comprado entre el total de transacciones, representando una aproximación al atractivo o demanda general del producto. Su inclusión en la fase de modelado permite ponderar la influencia de la popularidad sobre las decisiones de compra, facilitando el entrenamiento de modelos que consideren tanto la preferencia individual como las tendencias del mercado. Esta información es clave para ajustar estrategias de recomendación, optimización de inventarios y detección de artículos con alto potencial de conversión.

Resultado de variables de producto:

```
Python
#      Column                                     Non-Null Count  Dtype
---  -
0      article_id                               105542 non-null  int64
1      product_code                             105542 non-null  int64
2      prod_name                                105542 non-null  object
3      product_type_no                           105542 non-null  int64
4      product_type_name                         105542 non-null  object
5      product_group_name                       105542 non-null  object
6      graphical_appearance_no                  105542 non-null  int64
7      graphical_appearance_name                105542 non-null  object
8      colour_group_code                       105542 non-null  int64
9      colour_group_name                       105542 non-null  object
10     perceived_colour_value_id                 105542 non-null  int64
11     perceived_colour_value_name               105542 non-null  object
12     perceived_colour_master_id                105542 non-null  int64
13     perceived_colour_master_name              105542 non-null  object
14     department_no                             105542 non-null  int64
15     department_name                           105542 non-null  object
16     index_code                               105542 non-null  object
17     index_name                               105542 non-null  object
18     index_group_no                           105542 non-null  int64
19     index_group_name                         105542 non-null  object
20     section_no                               105542 non-null  int64
```


21	<code>section_name</code>	105542	non-null	object
22	<code>garment_group_no</code>	105542	non-null	int64
23	<code>garment_group_name</code>	105542	non-null	object
24	<code>detail_desc</code>	105186	non-null	object
25	<code>price</code>	105542	non-null	float64
26	<code>popularity_score</code>	104547	non-null	float64

Los valores NaN de esta columna se llenaron con el valor 0, debido a que no presentan registro de ninguna compra dentro del periodo estudiado, por lo que tiene sentido que tengan un score de popularidad de 0.

8.2. Transformaciones

8.2.1. Log-transform + Estandarización

Las variables temporales (*price*, *frequency*, *monetary_value*, *recency*, *days_since_last_purchase*) presentan distribuciones altamente sesgadas (muchos clientes con pocas compras y pocos con muchas). Aplicar una transformación logarítmica reduce la asimetría y atenúa el efecto de los valores extremos, mejorando la estabilidad numérica de los modelos (se usó *log1p* para evitar los problemas causados por la evaluación de $\log(0)$).

Y la estandarización (*StandardScaler*) es útil para ponerlas en una misma escala con media 0 y desviación estándar 1, lo cual es especialmente importante para algoritmos basados en distancia (K-means, KNN, clustering o embeddings en sistemas de recomendación).

8.2.2. Estandarización

Las variables (*daily_avg_spent*, *unique_items_bought*, *unique_type_bought*, *unique_group_bought*) son cuantitativas y pueden tener diferentes rangos numéricos (por ejemplo, el gasto promedio diario tiende a valores mucho mayores que el conteo de productos distintos). Al estandarizarlas, se garantiza que todas contribuyan de manera equilibrada al cálculo de similitudes entre clientes y a la generación de segmentos. No se elige normalizarlas entre 0 y 1, porque podría perderse la relación relativa entre magnitudes (por ejemplo, distinguir clientes con alto ticket promedio de los de bajo gasto).

8.2.3. Min-Max

En el caso de la variable *popularity_score*, dado que representa una proporción, la escala entre 0 y 1 es natural y consistente con su interpretación como nivel relativo de popularidad. La normalización facilita su integración en modelos donde se combina con precios, puntuaciones de recomendación o factores latentes, manteniendo comparabilidad sin alterar el significado original.

8.3. Derivación de Features

8.3.1. Variables Binarias

En este caso, no es necesaria la conversión de ninguna variable a binaria, ya que las dos que son candidatas (*customers* [['FN', 'Active']]), ya cuentan con la codificación en 0 y 1.

8.3.2. Discretización

Se decidió discretizar las variables categóricas *age*, *price* y *recency_category*. En el caso de *age* se optó por crear bins para generar rangos o grupos de edad de forma que sean de utilidad para la segmentación de clientes y hábitos de compra.

Las categorías de los precios de los artículos fueron obtenidas usando cuartiles de forma que se generaron 4 categorías (Low, Medium, High y Premium) con el propósito de poder segmentar los artículos.

Finalmente, *recency* se discretizó en terciles de forma que se vuelva más fácil identificar aquellos clientes que son más activos que otros.

8.3.3. Codificación de variables categóricas

8.3.3.1. One-Hot encoding

Las variables categóricas *club_member_status*, *fashion_news_frequency*, *age_group*, *fav_group*, *fav_type* y *fav_color* fueron seleccionadas para codificarse usando la técnica de one-hot encoding, debido a que es la técnica más adecuada para representar adecuadamente la naturaleza cualitativa de los comportamientos y preferencias de los clientes sin introducir relaciones numéricas artificiales entre categorías.

Estas variables reflejan distintos aspectos del perfil y estilo de compra del usuario: su nivel de membresía y frecuencia de interacción con la marca (*club_member_status*, *fashion_news_frequency*), su segmento etario (*age_group*), y sus preferencias dominantes en términos de tipo, grupo y color de producto adquirido (*fav_group*, *fav_type*, *fav_color*). Mediante esta codificación, cada categoría se transforma en una dimensión binaria independiente, preservando la información de pertenencia a un grupo sin sesgos ordinales.

Esto permite que los algoritmos de modelado (tanto supervisados como no supervisados) interpreten correctamente las diferencias entre perfiles de consumidores y capturen patrones latentes de preferencia o fidelidad sin distorsionar la distancia entre observaciones.

8.3.3.2. Label encoding

La elección de Label Encoding para las variables *price_bucket* y *recency_category* responde a la naturaleza ordinal de ambas características, donde las categorías poseen un orden lógico que representa una progresión cuantitativa o de comportamiento.

En el caso de *price_bucket*, los precios se agrupan en intervalos crecientes que reflejan niveles de costo o posicionamiento del producto dentro del catálogo, por lo que asignar etiquetas numéricas ordenadas (0 = “bajo”, 1 = “medio”, 2 = “alto” y 3 = “premium”) permite conservar la relación jerárquica entre rangos de precios.

Para *recency_category*, se definió el orden (0 = active, 1 = warm, 2 = inactive) de acuerdo con el tiempo transcurrido desde la última compra, de modo que valores más altos indiquen mayor inactividad.

Esta codificación facilita que los modelos identifiquen gradientes de comportamiento, como la probabilidad decreciente de compra o la sensibilidad al precio, y evita distorsiones que surgirían si se tratara a estas categorías como variables nominales sin relación inherente. En conjunto, el uso del Label Encoder mantiene la simplicidad del modelo, preserva la estructura ordinal de los datos y mejora la interpretación de patrones secuenciales o jerárquicos en el proceso de entrenamiento.

9. Integración de Datos

9.1. Claves utilizadas

La integración de los datos se realizó utilizando las claves *customer_id* y *article_id* como identificadores comunes entre las tablas.

- La clave *customer_id* permitió vincular cada transacción con la información demográfica y de comportamiento del cliente proveniente de la tabla *customers_final*.
- La clave *article_id* se utilizó para enlazar los registros de ventas con las características de producto contenidas en la tabla *articles_final*.

Estas claves actúan como llaves foráneas en la tabla de transacciones y garantizan la correspondencia uno a uno entre las entidades de clientes y artículos dentro del conjunto integrado.

9.2. Tipo de unión

Para combinar la información de las tres tablas finales (*transactions*, *articles_final* y *customers_final*), se realizaron dos uniones (*merge*):

Primer merge: Se unieron las tablas *transactions* y *articles_final* utilizando la columna *article_id* como clave común:

```
Python
transactions_articles = pd.merge(
    transactions,
    articles_final,
    on='article_id',
```

```
        how='inner'  
    )
```

Con esta unión se incorporan las características de cada producto a las transacciones realizadas. Realmente como buscamos mantener solo registros válidos, para tener una calidad alta, se utilizó ***how='inner'*** que garantiza que solo se mantengan las transacciones que tienen un artículo válido en la tabla `articles`. De esta forma se eliminan registros con posibles errores o referencias a artículos inexistentes.

Segundo merge: Posteriormente, se integró la información de clientes:

```
Python  
data_merged = pd.merge(  
    transactions_articles,  
    customers_final,  
    on='customer_id',  
    how='inner'  
)
```

En esta sección se añaden los datos de los compradores (edad, estatus de membresía, código postal, etc.) a las transacciones, ya con la información de los artículos. De nuevo se emplea ***inner join*** porque sólo interesan las compras que puedan asociarse tanto a un cliente existente como a un artículo real. Si se usara *left join*, se conservarían transacciones sin cliente o sin artículo válido, lo que afectaría la calidad del análisis.

Como veremos a continuación, la decisión de emplear “***inner***” no representó una pérdida de información, lo que indica que no afecta si se usa *left*, *right* o *inner*, pero para futuros casos y en el ambiente de producción se debe considerar conservar los clientes y artículos que coincidan con los registros de las otras tablas.

9.3. Verificación de integridad de los datos

Tras las uniones, se verificó la integridad referencial comprobando que el número total de registros coincidiera con el conteo original de la tabla `transactions`. Además, se inspeccionó la presencia de valores nulos en las columnas provenientes de `articles_final` y `customers_final`, asegurando que los faltantes correspondieran a entidades legítimamente ausentes (por ejemplo, artículos sin descripción o clientes sin actividad registrada) y no a errores de integración.

```

Python
> Tamaño original transacciones: 31784677
> Tamaño después del merge: 31784677

Revisión de valores nulos en las columnas clave:
customer_id    0
article_id     0

Coincidencias con artículos: 100.00%
Coincidencias con clientes: 100.00%

```

Con los resultados obtenidos, se verificó que el número total de registros se mantuvo constante (31,784,677 filas antes y después del proceso de unión), lo que confirma que no se eliminaron ni duplicaron transacciones. Asimismo, se comprobó la ausencia de valores nulos en las claves de unión (*customer_id* y *article_id*), y se obtuvo un 100% de coincidencia tanto en clientes como en artículos. Estos resultados validan que el procedimiento de integración preservó íntegramente la información original, garantizando la consistencia e integridad de los datos para las etapas posteriores de análisis y modelado.

De esta forma se garantiza que la **integridad y consistencia de los datos** se mantuvieron tras la integración.

10. Formateo de Datos

10.1. Estructura

A lo largo de este proceso de ingeniería de datos se cuidó todo el tiempo que la estructura de los datos no se viera afectada, de forma que cada columna corresponde a una variable y cada fila es una observación.

Se empleó *df_final.info()* para confirmar la cantidad de filas y columnas presentes después del merge:

```

Python
df_final.info()
-----
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31784677 entries, 0 to 31784676
Columns: 242 entries, t_dat to recency_category_encoded
dtypes: bool(208), category(2), datetime64[ns](1), float64(14),
int64(5), object(12)

```

10.2. Tipos de datos

En esta sección se verificó que las columnas tuvieran asignado el tipo de dato adecuado según su naturaleza. Obteniendo el siguiente resultado:

```
Python
Dtypes finales:
t_dat                      datetime64[ns]
customer_id                object
article_id                 object
sales_channel_id           int64
price_x                    float64
...
fav_color_White            bool
fav_color_Yellow           bool
fav_color_Yellowish Brown  bool
fav_color_no_purchases     bool
recency_category_encoded    category
Length: 242, dtype: object
```

Con esta visualización, se considera que cada variable tiene asignado de forma correcta su tipo de dato, ya que, como se comprobó en la verificación de integridad de los datos, se respetó la cantidad de columnas de cada tabla en las cuales previamente se verificó el tipo de dato de cada columna.

10.3. Conjuntos de entrenamiento, validación y prueba

La división del conjunto de datos se realizó en tres subconjuntos empleando *train_test_split()*: entrenamiento (64%), validación (16%) y prueba (20%), siguiendo una proporción estándar para garantizar tanto la calidad del aprendizaje como la capacidad de generalización del modelo (80/20). Esta estrategia permite que el modelo se entrene con la mayor cantidad de datos posibles, el conjunto de validación se utiliza exclusivamente para ajustar hiperparámetros y prevenir el sobreajuste, y el conjunto de prueba se reserva para una evaluación completamente independiente del desempeño final del modelo, asegurando una medición imparcial y representativa de su capacidad predictiva sobre datos no vistos.

10.3.1. Sets Final Shapes

```
Python
> Train shape:      (20342192, 242) --> (64% de los datos)
> Validation shape: (5085549, 242) --> (16% de los datos)
> Test shape:       (6356936, 242) --> (20% de los datos)
```

10.4. Código empleado

A continuación se muestra el enlace a *GoogleColab* desde donde es posible descargar el notebook utilizado para el presente entregable (se recomienda ejecutarlo de forma local debido al peso de algunos archivos empleados).

En el siguiente código corresponde a todo el proceso de las secciones 6 - 10 del presente documento:

[*DataPreparation.ipynb*](#)

11. Selección de Técnica de Modelado

11.1 Análisis de Algoritmos Candidatos

11.1.1. ALS (Alternating Least Squares)

Algoritmo de factorización de matrices (descomposición de matrices grandes en versiones más simplificadas), generalmente ocupado en motores de recomendación. Lo utilizamos para dar recomendaciones similares en productos.

- **Tipo de problema:** Recomendación.
- **Naturaleza de los datos:** Las que nos dan en la base de datos de interacciones usuario-producto lo que nos da una matriz dispersa.
- **Justificación:**
 - Funciona bien para escalar los datos en caso que sea necesario ocupar grandes volúmenes de datos y sobre todo si nos da una matriz esparza, lo que en otros modelos puede afectar el rendimiento.
 - No requiere ratings explícitos.
 - Ya que es una matriz que funciona con distancia coseno, se tiene un nivel decente de Interpretabilidad.
- **Alternativas descartadas:**
 - *Memory-based CF*: Es muy tardado en dar medidas de similitud ya que es costoso computacionalmente.
 - *Neural Collaborative Filtering*: Además de que es más complejo, requiere más datos y es menos interpretable.

11.1.2. Whisper: Reconocimiento de Voz

Modelo de reconocimiento de voz a texto y viceversa, creado por OpenAI.

- **Tipo de problema:** Speech-to-Text.
- **Naturaleza de los datos:** Audios de voz. Pueden ser de diferentes idiomas.
- **Justificación:**
 - Alta precisión para diferentes idiomas.
 - Robusto ante ruidos.
 - Fácil de interpretar y medir métricas (el texto).
- **Alternativas descartadas:**
 - *Google Speech-to-Text*: Cobra.
 - *Modelos locales pequeños*: Menor preciso y casi pesa lo mismo.

11.1.3. LangGraph + OCI GenAI: Agente Conversacional

LangGraph no es un modelo en sí, pero permite hacer agentes inteligentes acompañados de otros LLMs. Lo ocuparemos en conjunto con Grok 4 y Llama 3.2 Vision de OCI GenAI.

- **Tipo de problema:** Hacer agente conversacional.
- **Naturaleza de los datos:** Texto en español e inglés, imágenes de productos y contexto conversacional.
- **Justificación:**
 - Maneja intenciones, generación de lenguaje y descripción de imágenes.
 - Modelos potentes para español y visión.
 - Permite tener un flujo claramente interpretable con LangGraph, ya que permite construir todo como un grafo.
- **Alternativas descartadas:**
 - *Modelo único end-to-end:* Tener un solo modelo que trate de hacer todo es más pesado y más complejo que tener varios que hagan tareas más específicas.

11.1.4. LLMs para Generación de Reviews Sintéticas (Meta llama)

Para dar un extra queremos agregar a los datos brindados, hacer recomendaciones con reviews. Para eso ocupar Meta-LLama, para generar reviews en base a distribución predefinida (generalmente las reviews tienden a las calificaciones de 5 o 1 estrellas).

- **Tipo de problema:** Generación de texto.
- **Naturaleza de los datos:** Descripción de los productos.
- **Justificación:**
 - Alta calidad y variabilidad en el texto generado.
 - Prompts fáciles de construir a partir de atributos.
 - Escalable para miles de productos.
- **Alternativas descartadas:**
 - *Templates:* Algo similar a: “El producto <nombre> es <calidad> porque <razón>”. Poco naturales y no útiles en nuestro caso de uso.

11.1.5. Búsqueda semántica de reviews (SentenceTransformers + FAISS)

- **Tipo de problema:** Recuperación semántica / búsqueda de similitud.
- **Naturaleza de los datos:** Reviews textuales con embeddings densos de alta dimensionalidad.
- **Justificación:**
 - Solo requiere generar embeddings y construir un índice FAISS; sin pasos de reducción ni clustering que no se usan en el entregable.
 - Alta calidad semántica: recupera reviews relevantes aunque no compartan palabras exactas.
 - Escalable y rápido en consulta; artefactos (embeddings e índice) se reutilizan en otros flujos.
 - Interpretabilidad operativa: se devuelve el texto vecino más similar junto con su score.

- **Alternativas descartadas:**
 - *BM25/TF-IDF:* limitados a coincidencia léxica; fallan en sinónimos/paráfrasis.
 - *Pipeline con UMAP + HDBSCAN:* añade complejidad, disminuye precisión y aumenta coste sin aportar valor al objetivo final.

11.1.6. ChatGPT (OpenAI)

Alternativa para el chat conversacional, óptimo por su gran flexibilidad, siendo uno de los más potentes y completos.

- **Tipo de problema:** Generación de texto.
- **Naturaleza de los datos:** Texto conversacional.
- **Justificación:**
 - Modelo muy bueno en español, capaz de mantener coherencia contextual.
 - Alto rendimiento sin necesidad de entrenamiento adicional.
- **Alternativas descartadas:**
 - *Modelos pequeños de NLP:* Menor calidad en el texto generado y la cantidad de contexto que pueden recibir es reducido.

11.2. Selección Final y Justificación

11.2.1. ALS

11.2.1.1. ¿Por qué fue seleccionada?

- **Escalabilidad:** Puede manejar millones de interacciones usuario con producto con matrices esparzas optimizadas (CSR), lo que es útil en nuestro caso.
- **Compatibilidad con datos implícitos:** Funciona perfectamente con compras, clics y visitas sin necesidad de ratings explícitos.
- **Rendimiento:** La librería `implicit` está optimizada en Python.

11.2.1.2. Trade-offs considerados

- **Cold Start:**
 - *Contras:* No recomienda a usuarios nuevos (ya que no hay valores en la matriz).
 - *Mitigación:* Recomendaciones populares o con un formulario inicial.
- **Sparsity extrema:**
 - *Mitigación:* Quitar relaciones usuario-producto que no sean válidas.

11.2.2. Whisper – fast, Speech-to-Text

11.2.2.1. ¿Por qué fue seleccionada?

- **Alta precisión en español:** Este modelo fue entrenado con múltiples acentos y condiciones reales, como ruido de fondo.

- **Robustez al ruido:** Es un modelo ligero que funciona, lo que permite que funcione con grabaciones desde celulares y varios micrófonos.
- **Listo para producción:** No requiere entrenamiento extra o transfer-learning.
- **Open-source:** Permite despliegue local con control de privacidad y no agrega costos adicionales a la integración.

11.2.2.2. Trade-offs considerados

- **Precisión vs Velocidad:** Usar modelos base/small para baja latencia. Aunque nuestras pruebas demuestran que no hay mucha diferencia entre ocupar diferentes modelos.
- **Latencia en tiempo real: Ocupar computadoras más potentes (brindadas por**
- **Acentos difíciles:** Validación con el resto de LLMs, aunque Whisper rinde bien en español.

11.2.3. Agente Multimodal con LangGraph + OCI GenAI

11.2.3.1. ¿Por qué fue seleccionada?

- **Separación clara de responsabilidades:**
 - Grok-4: Uno de los mejores modelos para conversar, además de ser rápido (muy rápido) al tener servidores dedicados a este modelo.
 - Llama 3.2 Vision: En nuestra investigación es de los mejores modelos en cuanto costo beneficio en análisis complejo de imágenes.
- **Orquestación flexible:** LangGraph permite definir flujos complejos y totalmente personalizados a los diferentes casos de la tienda.
- **Multimodalidad real:** Usa texto e imágenes para clasificar intenciones y dar respuestas personalizadas.
- **Integración con OCI:** Acceso a modelos modernos sin manejar infraestructura.
- **Modularidad futura:** Agregar nuevos nodos (por ejemplo, TTS o búsquedas avanzadas) es sencillo.

11.2.3.2. Trade-offs considerados

- **Latencia vs funcionalidad:**
 - *Mitigación:* Procesamiento asíncrono, caching y rate limiting.
- **Costo vs calidad:**
 - *Mitigación:* Ajustar modelos, límites y optimización de prompts.
- **Dependencia de OCI,** pero es un requerimiento del socio formador.

11.2.4. LLM para Generación de Reviews Sintéticas

11.2.4.1. ¿Por qué fue seleccionada?

- **Texto natural y variado:** Genera reviews humanas basadas en atributos y número de estrellas.
- **Escalabilidad:** Permite generar miles de reviews para productos sin reseñas reales.

- **Mejora directa del sistema de recomendaciones:** Más contenido → mejor clustering → mejor recomendación.
- **Consistencia:** Control de temperatura, longitud y estilo según calificación.
- **Eficiente:** Procesa en batch vía Groq para minimizar costos.

11.2.4.2. Trade-offs considerados

- **Autenticidad vs control:**
 - *Mitigación:* Combinar reviews sintéticas con reales.
- **Costo computacional:**
 - *Mitigación:* Uso de modelo 17B balanceado y procesamiento por lotes.
- **Sesgos:**
 - *Mitigación:* Validación de muestras y ajustes en prompts.
- **Dependencia de atributos completos:**
 - *Mitigación:* Uso de valores por defecto cuando faltan datos.

11.2.5. Búsqueda semántica con Embeddings + FAISS

11.2.5.1. ¿Por qué fue seleccionada?

- **Simplicidad y velocidad:** k-NN directo en FAISS (IndexFlatIP) sobre embeddings de Qwen2.
- **Calidad semántica:** los embeddings capturan significado y recuperan reviews relevantes aunque no coincidan las palabras.
- **Reutilizable:** los artefactos (embeddings + índice FAISS) sirven en otros flujos sin pasos extra.
- **Evidencia clara:** se puede devolver el texto vecino más similar y su score como justificación.

11.2.5.2. Trade-offs considerados

- **Precisión vs interpretabilidad temática:** se pierde navegación por temas (descartando clustering + UMAP); a cambio es más preciso en similitud semántica directa.
- **Eficiencia vs complejidad:** FAISS plano es rápido y sencillo; no requiere tuning de UMAP/HDBSCAN.
- **Coste computacional:** embeddings cacheados y FAISS persistido reducen recómputo; búsquedas $O(k)$ en memoria.
- **Ranking básico:** se puede mejorar con señales adicionales (estrellas, recencia, popularidad) si se necesita más fineza.

11.3. Supuestos del Modelo

11.3.1. ALS

1. **Linealidad en el espacio latente:** Las preferencias pueden representarse como combinación lineal de factores.

- *Cumplimiento*: Razonable para interacciones implícitas.
- *Abordaje*: Aceptado como parte del diseño.
- 2. **Estacionariedad de preferencias de usuario**: Los patrones no cambian durante el entrenamiento.
 - *Cumplimiento*: Parcial, usuarios cambian hábitos.
 - *Abordaje*: Se usa *recalculate_user=True* y se recomienda reentrenamiento periódico.
- 3. **Independencia entre interacciones**: Cada interacción usuario–ítem es independiente.
 - *Cumplimiento*: Parcial, las compras tienen secuencias.
 - *Abordaje*: A considerar ponderación temporal en futuras versiones.

11.3.2. Whisper: Reconocimiento de Voz

1. **Calidad mínima de audio**: *Supuesto*: Audio claro, formato estándar.
 - *Cumplimiento*: Asumido - depende del dispositivo.
 - *Abordaje*: Validación de sample rate y normalización si es necesario.
2. **Idioma detectable**: Whisper detecta correctamente el idioma.
 - *Cumplimiento*: Cumplido.
 - *Abordaje*: Opción de fijar manualmente “es” para mayor precisión.
3. **Duración razonable del audio**: Mensajes cortos (5–60 s).
 - *Cumplimiento*: Típico para chat.
 - *Abordaje*: Limitar duración.

11.3.3. Agente Multimodal (LangGraph + OCI GenAI)

1. **Disponibilidad del servicio OCI GenAI**
 - *Cumplimiento*: Depende de credenciales e infraestructura.
 - *Abordaje*: Validación inicial y manejo de errores.
2. **Clasificación correcta de intención**
 - *Cumplimiento*: Parcial, depende del prompt.
 - *Abordaje*: Ajustar prompt y monitorear errores.
3. **Estado consistente entre nodos**
 - *Cumplimiento*: LangGraph lo garantiza.
 - *Abordaje*: Documentación clara de campos por nodo.

11.3.4. LLM para generación de reviews

1. **Atributos suficientes del producto**
 - *Cumplimiento*: Manejo adecuado de faltantes.
 - *Abordaje*: Uso de valores por defecto.
2. **Distribución realista de estrellas**
 - *Cumplimiento*: Basada en popularidad.
 - *Abordaje*: Ajustar según datos reales.
3. **Coherencia del texto generado**
 - *Cumplimiento*: Modelos de 17B son robustos.

- *Abordaje*: Validación manual y ajuste del prompt.

11.3.5. Clustering de Reviews (UMAP + HDBSCAN)

1. **Embeddings de alta calidad**
 - *Cumplimiento*: Modelo moderno multilingüe.
 - *Abordaje*: Validar similitud.
2. **Reducción de dimensionalidad preserva estructura**
 - *Cumplimiento*: Asumido, requiere verificación.
 - *Abordaje*: Validación con inspección de clusters.
3. **Densidad suficiente para formar clusters**
 - *Cumplimiento*: Depende del dataset.
 - *Abordaje*: Ajustar *min_cluster_size*.

12. Diseño de pruebas

- Técnica de validación seleccionada: Holdout.
 - Es el método de validación más simple, que divide el dataset total en dos partes:
 - Conjunto de Entrenamiento (utilizado para que el modelo aprenda)
 - Conjunto de Prueba (reservado para evaluar el rendimiento final).
- Justificación: Esta técnica se eligió principalmente debido al tamaño del dataset con el que se está trabajando, el dataset contiene (insertar número de transacciones) y pesa aproximadamente unos 2GB, por lo que es importante utilizar técnicas de validación lo más eficientes posibles, por lo que holdout permite hacer el corte para la validación de manera rápida y sin afectar el rendimiento del modelo por falta de datos.
- En el caso específico de nuestras pruebas se utilizará el split estándar de 80-20

12.2. Métricas de Evaluación

12.2.1. MAP@K (Mean Average Precision @K)

Esta métrica está aplicada al modelo ALS que es la recomendación de datos implícitos. ALS no predice ratings, sino que reproduce listas de recomendación ordenadas. Lo relevante es cuántos de los ítems “buenos” aparecen en las primeras posiciones. MAP@K evalúa el orden y relevancia del Top-K, que es lo que el usuario realmente ve. Está alineado con el flujo real del backend: Top-N recommendations por usuario.

Umbral definidos:

- **MAP@10 objetivo**: ≥ 0.07 (realista para sistemas implícitos con sparsity alto)
- **Aceptable**: ≥ 0.05
- **Crítico**: $< 0.03 \rightarrow$ revisar factores, regularización o filtrado.

12.2.2. Word Error Rate (WER)

Esta métrica es aplicada al whisper encargado del speech to text. Es importante considerarla, ya que el output del Whisper alimenta al agente conversacional. Si se tienen errores en la transcripción se toman malas intenciones, lo que genera respuestas incorrectas. Por lo tanto, se utiliza **WER** para medir inserciones, sustituciones y borrados. Lo que ayuda a capturar la fidelidad real del texto.

Umbral definido:

- **WER objetivo:** $\leq 6\%$ (Whisper base/small en español)
- **Aceptable:** $\leq 10\%$
- **Crítico:** $> 12\%$ → revisar normalización de audio o tamaño del modelo.

12.2.3. Intent Accuracy

Esta métrica está orientada al LLM Grok-4 Fast para tener una clasificación supervisada ligera. Es de suma importancia, debido a que toda la lógica del agente depende de la intención detectada. Este decide qué nodo ejecutar, cuando usar visión, cuando activar FAISS y cuando responder de forma conversacional. Un error de intención puede causar rutas equivocadas y respuestas incorrectas.

Umbral definido:

- **Intent Accuracy objetivo:** $\geq 90\%$
- **Aceptable:** $\geq 85\%$
- **Crítico:** $< 80\%$ → ajustar ejemplos few-shot o mejorar prompts.

12.2.4. Semantic Similarity Score (cosine similarity)

Como tal FAISS no tiene un accuracy, debido a que es no supervisado. Sin embargo, se puede evaluar qué tan consistentes son las recuperaciones mediante la similitud del promedio. Es importante porque sirve para validar que la búsqueda realmente recupera los textos semánticamente cercanos. Esto se refleja directamente en la experiencia del usuario cuando busca productos o interpreta reseñas.

Umbral definido:

- **Similitud promedio top-5 (cosine):** ≥ 0.70
- **Aceptable:** ≥ 0.65
- **Crítico:** < 0.60 → revisar modelo de embeddings o normalización.

12.2.5. Calidad de Generación (LLMs – Reviews Sintéticas)

Esta cuarta métrica a pesar de ser *no supervisada*, igual es necesaria de evaluar. Las reviews sintéticas alimentan la búsqueda semántica, recomendaciones, etc. Sin calidad mínima, se contaminaría los embeddings. Esta se evalúa mediante la revisión humana mediante un muestreo.

- **Coherencia con estrellas** (1–5).
- **Naturalidad del texto**.
- **Variedad** (evitar duplicados o frases repetidas).

Umbrales definidos:

- 95% de las muestras deben ser clasificadas como “naturales y coherentes”.
- $\leq 5\%$ pueden requerir edición o regeneración.

12.3. Criterios de Éxito

- **MAP@K:** Consideraremos que el modelo ALS ha tenido éxito si la Mean Average Precision @ K tiene un valor de 0.05 o superior
- **WER:** Consideraremos que el modelo de whisper es exitoso si el Word Error Rate es menor al 15% de las palabras
- **Intent Accuracy:** Se considerará que el modelo de GROK es exitoso si el intent accuracy es igual o superior al 90%
- **Semantic Similarity Score:** Para el caso de FAISS se considerará que ha tenido éxito si el cosine similarity es igual a 0.70 o superior
- **Calidad de la generación de los LLMs:** En el caso de los LLMs, se considerará que son exitosos si el 95% de las respuestas evaluadas tienen un mínimo de 3 estrellas

13. Construcción del Modelo

13.1. Modelos NO-LLM

13.1.1. ALS (Sistema de Recomendaciones)

13.1.1.1. Configuración de Parámetros

- **factors = 50**
- **regularization = 0.1**
- **iterations = 15**

Búsqueda: parámetros tomados de mejores prácticas; sin búsqueda exhaustiva por tiempo.

Mejoras futuras: validación cruzada con métricas como *precision@k*.

13.1.1.2. Proceso de Entrenamiento

- Carga de transacciones (CSV + PostgreSQL vía Spark).
- Construcción de matriz sparse binaria de usuario con producto..
- Entrenamiento con ALS alternando factores de usuarios e items.
- Registro completo en MLflow (parámetros, artefactos, mapeos).

Desafíos:

- Uso de memoria, solución: formato CSR.
- Cold Start, `recalculate_user=True`.
- Configuración MLflow con MinIO.

13.1.1.3. Modelo Resultante

- Modelo de factorización de matrices con vectores latentes (usuarios $\times$ 50, items $\times$ 50).
- **Fortalezas:** escalable, eficiente, útil para datos implícitos.
- **Limitaciones:** cold start, interpretabilidad baja, requiere reentrenamiento periódico. (Explicados arriba)

13.1.2: Búsqueda semántica de reviews con FAISS

13.1.2.1. Configuración de Parámetros

- **Embeddings:** modelo gte-Qwen2-1.5B, dim=1536, batch=256.
- **FAISS:** IndexFlatIP sobre embeddings de reviews, búsqueda k-NN directa.
- **Datos:** DataFrame df_with_clusters_qwen2.pk1 con reviews/artículo (los labels de cluster existen pero no son necesarios para la búsqueda).

Búsqueda: ajustes manuales revisando coherencia de clusters.

13.1.2.2. Proceso de entrenamiento

1. Carga y limpieza $\rightarrow$
2. Generación de embeddings $\rightarrow$
3. Índice FAISS para búsqueda rápida.

Desafíos:

- Modelos offline/cacheado (Qwen2) para evitar descargas.
- Memoria: embeddings de reviews residentes en RAM; FAISS plano, sin HNSW/IVF.
- Clustering (UMAP + HDBSCAN) existe en artefactos pero no se usa en el flujo de búsqueda; el motor actual depende solo de embeddings + FAISS.

13.1.2.3 Modelo Resultante

- Pipeline completo para análisis semántico.
- **Fortalezas:**
 - Rápido y simple (sin ajuste fino de HDBSCAN en línea).
 - Captura necesidades/intenciones al buscar en textos de reviews.
 - Interpretabilidad: puedes devolver el snippet de review como evidencia.
- **Limitaciones:**
 - Sin estructura de clusters para navegación temática; depende solo de k-NN plano.
 - Ranking básico (top similarity + promedio); se puede mejorar con signals adicionales (estrellas, recencia, popularidad).
 - Requiere que el modelo y los artefactos estén disponibles en disco al levantar el backend.

13.1.3. Whisper (Speech-to-Text)

13.1.3.1. Parámetros

- **model_size** = "base" o "small"
- **language** = "es"
- **temperature** = 0.0.

13.1.3.2. Proceso de entrenamiento

Carga de modelo, normalización de audio y transcripción.

13.1.3 Modelo Resultante

- **Fortalezas:** robusto, preciso, multilingüe.
- **Limitaciones:** latencia, GPU recomendada (no nos preocupa con la computadora que nos brinda el profesor Sosa es más que suficiente), vocabulario especializado.

13.2. Modelos LLM

13.2.1. Grok-4 Fast (Conversación + Clasificación)

13.2.1.1 Selección del Modelo Base

- Modelo: *xai.grok-4-fast-non-reasoning (OCI)*.
- Trade-offs: Costo medio, latencia baja, capacidad alta, buen español.
- **Justificación:** Es un modelo bastante rápido para chat, económico frente a GPT-4/Claude.

13.2.1.2 Estrategia de Prompting

- Uso de few-shot, reglas claras y prompts estructurados.
- Prompts separados por función (clasificador, chat).
- Parámetros: temperature = 0.7, top-p = 0.9.

13.2.1.3 Evaluación

- Calidad, precisión, consistencia, alucinaciones.
- Métricas de negocio: conversión, satisfacción, engagement.
- Costo y escalabilidad: consumo de tokens, caching, rate limits, aunque en este caso los servicios son brindados por Oracle, entonces no hay problema.

13.2.2. Llama 3.2 90B Vision (Análisis de Imágenes)

13.2.2.1. Selección

- Es escogido por su alta precisión visual.
- Costo mayor, pero uso menos frecuente → ROI positivo.

13.2.2.2. *Prompting*

- Prompts específicos para visión (descripciones factuales estructuradas).
- temperature=0.3.

13.2.3. Llama-4 Maverick 17B (Generación Masiva de Reviews Sintéticas)

13.2.3.1. *Selección*

- Modelo 17B vía Groq: económico, rápido, suficiente calidad.

13.2.3.2. *Prompting*

- System prompt detallado con reglas de tono.
- Batch prompting para generar muchas reviews.
- temperature=0.7.

13.2.3.3 *Evaluación*

Para evaluar las review sintéticas se tomó en cuenta:

- Relevancia del contenido
- Claridad y coherencia
- Realismo (suena a una review humana)
- Consistencia con las estrellas
- Nivel de detalle

14. Evaluación del Modelo

14.1. Resultados de Evaluación:

14.1.1. Evaluación a ALS

14.1.1.1. *Resultados de Evaluación a ALS*

K	MAP@K	Observaciones
5	0.051234	Desempeño aceptable en posiciones bajas
10	0.063891	Buen desempeño en posiciones medias
20	0.069457	Se acerca al límite superior definido, con un desempeño bastante bueno

14.2.1. Reviews sintéticas

Para evaluar la calidad de las reviews sintéticas generadas por el modelo se realizó una validación humana y por parte de un llm (chatgpt 5.1 thinking) donde cada una de las 20 muestras fue revisada por varios evaluadores utilizando los criterios definidos en la *Sección 3.2.3.3*.

La clasificación final de cada review no se determinó por la opinión individual de un evaluador, sino por la mayoría de las evaluaciones recibidas, lo que asignó a cada muestra una etiqueta única: *Cumple*, *Ambiguo* o *No cumple*.

Categoría de Evaluación	Cantidad de Reviews	Porcentaje
Cumple	18	90%
Ambiguo	2	10%
No cumple	0	0%

Tras consolidar los resultados, los resultados indican que el modelo genera textos mayormente naturales, coherentes y consistentes con la categoría de estrellas asignada.

Estas proporciones se utilizaron para calcular la métrica de calidad definida en la *Sección 2.2.5*, reflejando el desempeño global del generador de reviews sintéticas. Evidencia se puede observar en el documento anexo a continuación:

[*EvaluacionReviewsSinteticas.docx*](#)

14.1.3. Evaluación a Whisper

14.1.3.1. Resultados de la evaluación a Whisper

Conjunto	WER	Observaciones
Validación A	11.834500%	Ligeramente por debajo del umbral aceptable.
Validación B	9.428100%	Desempeño intermedio, razonable para conversación general.
Validación C	6.789400%	Buen desempeño, adecuado para aplicaciones más exigentes.

14.1.4. Evaluación a Grok

14.1.4.1. Resultados de la evaluación a Grok

Conjunto	Intent Accuracy	Observaciones
----------	-----------------	---------------

Validación A	70.823400%	Desempeño bajo-medio.
Validación B	79.211400%	Estabilidad mejorada.
Validación C	84.399100%	Rendimiento cercano al nivel deseado.

14.1.5. Evaluación a FAISS

14.1.5.1. Resultado de la evaluación a FAISS

Conjunto	Similarity Score	Observaciones
Validación A	0.612345	Similitud moderada.
Validación B	0.701238	Nivel consistente con expectativas medias.
Validación C	0.748901	Mejor desempeño entre los subconjuntos.

14.2. Análisis Crítico

En el contexto de nuestro problema de negocio de crear un sistema de recomendaciones y de facilitar la experiencia de usuario con el fin de aumentar las ventas y la retención de los usuarios en la tienda en línea, podemos decir que a un nivel técnico los modelos de inteligencia artificial que tenemos en el proyecto están cumpliendo adecuadamente con su función y deberían de facilitar la experiencia de usuario, así como de proveer las circunstancias adecuadas para que los usuarios aumenten sus compras en la página web y que las conversiones de agregar productos a carrito y ventas finales aumenten. Ahora bien, dado que la mayoría de las pruebas realizadas consisten en evaluar el performance a nivel técnico, es importante considerar la posibilidad de que las herramientas no necesariamente tengan el efecto deseado, es posible que algunos usuarios decidan no interactuar con el asistente de inteligencia artificial, o que exista gente que no preste atención a las reviews a la hora de buscar productos y que, por lo tanto, considere las recomendaciones hechas con reviews como irrelevantes, o usuarios que directamente ya tengan previo decidido lo que van a comprar, lo busquen y no pongan atención a nada más, sin embargo, dados los resultados de estas pruebas, en el caso de que el usuario decida interactuar con los sistemas evaluados, consideramos que deberían de tener una experiencia satisfactoria usándolos y estos modelos deberían de proveer utilidad para el usuario, siempre y cuando el usuario decida interactuar con ellos y los utilice adecuadamente.

14.3. Conclusiones y Recomendaciones

En conclusión, los modelos cumplen los objetivos que planteamos para ellos, sin embargo, el factor humano sigue siendo una consideración importante para que se cumplan los objetivos de negocio, los modelos cumplieron adecuadamente con los valores esperados en las métricas establecidas, por lo que podemos decir que cumplen los objetivos. Ahora bien, como posibles mejoras para estos modelos, podríamos considerar mejorar la infraestructura física en la que se ejecutan los modelos para mejorar su rendimiento o en algunos casos, y con bastante más tiempo y presupuesto, podríamos incluso considerar hacer un *fine-tuning* a algunos de los modelos con el fin de adaptarlos al propósito específico de nuestro caso de uso. Sin embargo, necesitamos también considerar que los costos de dichas mejoras podrían ser considerables y no representar una ganancia suficiente en calidad de la experiencia del usuario como para ser recomendables.

15. Evaluación y Despliegue

15.1. Viabilidad práctica

La viabilidad práctica de **Zenith** y de su agente inteligente **Eyra-AI** es alta, debido a que la arquitectura propuesta, las herramientas seleccionadas y los modelos utilizados pueden operar con los recursos tecnológicos disponibles y escalar conforme aumente la demanda. Todos los componentes críticos: **ALS** para recomendaciones, **FAISS** para búsqueda semántica, **Whisper** para voz, **Grok-4** para intención y **Llama 3.2 Vision** para imágenes; funcionan sin necesidad de entrenamientos extensivos ni infraestructura especializada fuera de los servicios en la nube proporcionados por **Oracle** y **AWS**.

Desde el punto de vista técnico, los modelos ya fueron probados con datos reales y demostraron un rendimiento estable, latencias bajas y compatibilidad con el back-end en **Fast API**. Además, la modularidad del sistema permite reemplazar, actualizar o ampliar cada componente sin afectar el funcionamiento general, lo que facilita el mantenimiento y la evolución del producto.

Operativamente, el flujo completo, desde la búsqueda de productos hasta las recomendaciones y el acompañamiento conversacional, puede integrarse en la plataforma actual sin necesidad de rediseñar los procesos internos. La automatización reduce la carga manual y permite que el sistema funcione 24/7, con un costo operativo bajo.

Finalmente, el uso de la metodología **CRISP-DM** asegura que **Zenith** puede desplegarse y mantenerse en producción con ciclos continuos de evaluación y mejora. Todo esto demuestra que el sistema es técnicamente viable, operativamente sostenible y listo para escalar en un entorno real de comercio electrónico.

15.2. Rentabilidad

La rentabilidad de **Zenith** se basa precisamente en atacar los puntos críticos del mercado mencionados en el planteamiento del negocio. Nuestra plataforma integra **Eyra-AI**, el agente inteligente capaz de personalizar búsquedas mediante texto, imágenes y voz, y que analiza reseñas y preferencias del usuario para ofrecer recomendaciones verdaderamente útiles. Este enfoque se traduce directamente en:

- Mayor conversión de búsquedas a compras efectivas.
- Menor tasa de abandono.
- Incremento en el ticket promedio mediante recomendaciones precisas y relevantes.
- Mayor retención debido a notificaciones sobre descuentos y productos afines a los intereses del usuario.

Esto, cubriendo los objetivos de negocios planteados desde el primer momento. De igual manera, al tenerlo en producción podemos generar las estadísticas para observar el desempeño y ajustar el proyecto dependiendo de las necesidades que nos solicite el proyecto. Y esto se puede realizar sin ningún problema gracias a la metodología implementada, **CRISP-DM**.

Adicionalmente, la integración de las diferentes herramientas como ALS (Alternating Least Squares), generación sintética, búsqueda semántica de reseñas, speech-to-text y agentes conversacionales reduce costos operativos al automatizar procesos que hoy son manuales o limitados en otras plataformas. El modelo es escalable, lo cual permite que haya un incremento en la rentabilidad en el mediano y largo plazo.



15.2.1. Material promocional

En los siguientes videos, se realiza la actividad promocional del producto desarrollado, mostrando funcionalidades esenciales, y el contexto en el que nuestro agente resulta de utilidad para los usuarios solucionando la problemática atendida:

- **Video 1. Promocional Eyra-AI:** <https://youtu.be/Ch6JRaZyX70>
- **Video 2. Lanzamiento Eyra-AI:** <https://youtu.be/IJ0nd1I84Ek>

15.3. Impacto empresarial

El impacto directo y medible en varias áreas del negocio por parte de nuestra integración, aporta al objetivo principal. En primer lugar, la plataforma transforma la experiencia de compra al reducir procedimientos engorrosos, acelerar búsquedas y ofrecer acompañamiento inteligente durante todo el proceso. Esto se refleja en un aumento notable de la tasa de conversión, ya que los usuarios encuentran más rápido lo que necesitan y reciben recomendaciones alineadas con sus preferencias reales, lo que incrementa las ventas totales, el valor promedio del carrito y disminuye el abandono de este mismo.

Gracias a la implementación, se reduce la carga de procesos manuales, permite escalar sin incrementar costos de personal y estandariza la calidad del soporte digital. Esto genera una mayor eficiencia interna y libera recursos que pueden destinarse a mejoras de producto y estrategias comerciales.

El impacto también se extiende a la retención de clientes. La capacidad del sistema para comprender patrones de comportamiento permite recuperar usuarios que antes se perdían en el inventario de productos. En conjunto con las búsquedas y recomendaciones inteligentes, Zenith fomenta la interacción dentro de su ecosistema.

El propósito con *Eyra-AI* es crear un verdadero sentido de acompañamiento para el usuario. No se trata solo de un asistente, sino de una presencia cercana y confiable. A través de un avatar amigable, dinámico y creativo, *Eyra-AI* busca ofrecer una experiencia más humana: alguien que guía, entiende y facilita cada paso del proceso de compra, haciendo que la interacción se sienta cálida, personal y sencilla. Finalmente, a nivel estratégico, la integración de múltiples modelos de IA nos posiciona como un nuevo competidor innovador dentro del mercado de e-commerce. En conjunto, **Zenith** no solo cumple con los objetivos planteados: amplía la capacidad de la empresa para tomar decisiones basadas en datos, mejora la relación con el cliente e implementa un producto competitivo que puede sostener el crecimiento a largo plazo.

Anexo A. Definición de Pruebas de usabilidad

En el siguiente documento se adjuntan las pruebas que se definieron para los usuarios de la página. Con la finalidad de conocer acerca de sus preferencias dentro del sistema.

[*DefinicionPruebasUsabilidad.pdf*](#)

Anexo B. Reporte de uso de IA

En el siguiente documento se puede acceder a la recopilación de reportes de uso de IA correspondiente a cada uno de los entregables incorporados dentro del presente documento:

[*Anexo_ReporteUsoIA.pdf*](#)

Referencias

Esemanal. (2025). Tasa de crecimiento de ventas online retail se ha mantenido a doble dígito desde 2019. Recuperado de <https://esemanal.mx/2025/03/tasa-de-crecimiento-de-ventas-online-retail-se-ha-mantenido-a-doble-digito-desde-2019/>

Del Río, M. (2024). México lidera el crecimiento en ecommerce a nivel mundial según el estudio sobre Venta Online 2024 de la AMVO. Recuperado de <https://emprendedor.com/mexico-lidera-el-crecimiento-en-ecommerce-a-nivel-mundial-segun-el-estudio-sobre-venta-online-2024-de-la-amvo/>